

Pobranie danych i podział na train valid i test

```
In [2]: import os
import random
import numpy as np
import pandas as pd
pd.set_option('display.max_columns', None)
pd.set_option('display.max_colwidth', None)
pd.set_option('display.width', None)
# --- 1. Globalne ziarno losowości ---
SEED = 123
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)

# --- 2. Wczytanie i złączenie danych ---
trans = pd.read_csv("dane/train_transaction.csv")
ident = pd.read_csv("dane/train_identity.csv")

df = trans.merge(ident, on="TransactionID", how="left")

# --- 3. Podział Out-Of-Time (chronologiczny) ---
df = df.sort_values("TransactionDT").reset_index(drop=True)

n = len(df)
n_train = int(n * 0.60)
n_valid = int(n * 0.80) # 60% + 20%

train = df.iloc[:n_train]
valid = df.iloc[n_train:n_valid]
test = df.iloc[n_valid:]

# --- 4. Weryfikacja podziału ---
for name, part in [("train", train), ("valid", valid), ("test", test)]:
    print(f"{name:5s} | shape: {part.shape} | fraud rate: {part['isFraud'].mean()")
```

```
train | shape: (354324, 434) | fraud rate: 0.0338
valid | shape: (118108, 434) | fraud rate: 0.0390
test | shape: (118108, 434) | fraud rate: 0.0344
```

Preprocessing danych typowo pod sieci neuronowe bazując na EDA przeprowadzonym wcześniej

```
In [3]: # Wymaga sklearn >= 1.3 (TargetEncoder) i scipy
import joblib
from scipy.stats import skew
from sklearn.feature_selection import VarianceThreshold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (MinMaxScaler, OneHotEncoder,
```

```

TargetEncoder, FunctionTransformer)
from sklearn.impute import SimpleImputer

# State i grupy kolumn
TARGET = 'isFraud'
MISSING_THRESH = 0.80 # dla nie-V: odrzuć kolumny z NaN > 80%
V_CORR_THRESH = 0.05 # dla V_xxx: zachowaj jeśli |corr(isFraud)| ≥ 0.05
VAR_THRESH = 0.01 # odrzuć cechy numeryczne z wariancją < 0.01
SKEW_THRESH = 3 # signed_log1p tylko dla |skewness| > 3
CARD_THRESH = 15 # granica cat_low / cat_high

# Wykrywamy grupy kolumn ze struktury surowych danych
M_COLS = [c for c in train.columns if c.startswith('M') and c[1:].isdigit()]
D_COLS = [c for c in train.columns if c.startswith('D') and c[1:].isdigit()]
ID_COLS = [c for c in train.columns
            if c.startswith('id_') or c in ('DeviceType', 'DeviceInfo')]

# 1. Feature Engineering
def engineer_features(df):
    df = df.copy()

    # Czas z TransactionDT
    df['hour'] = (df['TransactionDT'] // 3600) % 24
    df['dayofweek'] = (df['TransactionDT'] // (3600 * 24)) % 7

    # Flag: email nadawcy == email odbiorcy (fraud rate 3x wyższy gdy True wg E
    df['email_match'] = (df['P_emaildomain'] == df['R_emaildomain']).astype(np.i

    # M1-M9: T->1, F->0, NaN->-1
    # NaN to odrębny stan semantyczny (niezweryfikowane), nie brakująca wartość
    for c in M_COLS:
        if c in df.columns:
            df[c] = df[c].map({'T': 1, 'F': 0}).fillna(-1).astype(np.float64)

    # Flagi missingness dla bloków D i identity:
    # sam brak wartości koreluje z fraudem (np. D7: 14.9% vs 2.7% wg EDA)
    for c in D_COLS + ID_COLS:
        if c in df.columns:
            df[f'{c}_isnan'] = df[c].isna().astype(np.int64)

    return df.drop(columns=['TransactionID', 'TransactionDT', TARGET], errors='i

X_train = engineer_features(train); y_train = train[TARGET].reset_index(drop=Tr
X_valid = engineer_features(valid); y_valid = valid[TARGET].reset_index(drop=Tr
X_test = engineer_features(test); y_test = test[TARGET].reset_index(drop=Tru

# 2. Filtracja kolumn
isnan_cols = [c for c in X_train.columns if c.endswith('_isnan')]
v_cols = [c for c in X_train.columns if c.startswith('V') and c[1:].isdigi
missing_rate = X_train.isna().mean()

# V_xxx: zachowaj jeśli NaN < 80% LUB |korelacja z isFraud| ≥ 0.05
# Ślepa filtracja 80% odrzuciłaby 23 kolumny V o korelacji 0.05-0.28 (wg EDA)
v_corr = X_train[v_cols].corrwith(y_train).abs()
v_keep = [c for c in v_cols
           if missing_rate[c] < MISSING_THRESH or v_corr.get(c, 0) >= V_CORR_THRE

# Pozostałe (bez V_xxx i flag _isnan): standardowy próg 80%
# _isnan flagi zawsze zachowujemy - mają 0 NaN i niosą sygnał
other_cols = [c for c in X_train.columns if c not in v_cols + isnan_cols]

```

```

other_keep = [c for c in other_cols if missing_rate.get(c, 0) < MISSING_THRESH]

keep_cols = v_keep + other_keep + isnan_cols
X_train, X_valid, X_test = X_train[keep_cols], X_valid[keep_cols], X_test[keep_cols]
print(f"Kolumny po filtracji: {len(keep_cols)} "
      f"(V_XXX: {len(v_keep)}, inne: {len(other_keep)}, _isnan: {len(isnan_cols)})")

# 3. Grupy zmiennych, wariancja, skośność
# "Stable" = kolumny celowo wykluczone z VarianceThreshold i signed_log1p:
# _isnan: binarne 0/1, semantycznie istotne nawet przy niskiej wariancji
# M_COLS: pre-zakodowane {-1, 0, 1} - nie mają sensu logować
stable = set(isnan_cols) | {c for c in M_COLS if c in X_train.columns}

num_all = X_train.select_dtypes(include=[np.number]).columns.tolist()
cat_all = X_train.select_dtypes(include=['object', 'string']).columns.tolist()

# Filtr wariancji (tylko nie-stable)
num_for_filter = [c for c in num_all if c not in stable]
selector = VarianceThreshold(threshold=VAR_THRESH)
selector.fit(X_train[num_for_filter])
low_var = [c for c, ok in zip(num_for_filter, selector.get_support()) if not ok]

X_train = X_train.drop(columns=low_var)
X_valid = X_valid.drop(columns=low_var)
X_test = X_test.drop(columns=low_var)
num_all = [c for c in num_all if c not in low_var]
print(f"Usunięto {len(low_var)} cech o niskiej wariancji -> {len(num_all)} cech")

# Skośność (tylko nie-stable) -> signed_log1p kompresuje ogon bez wykrzywania p
skewness = X_train[[c for c in num_all if c not in stable]].apply(lambda c: skewness[c])
num_skew = skewness[skewness.abs() > SKEW_THRESH].index.tolist()
num_normal = [c for c in num_all if c not in num_skew] # zawiera stable

cat_low = [c for c in cat_all if X_train[c].nunique() < CARD_THRESH]
cat_high = [c for c in cat_all if X_train[c].nunique() >= CARD_THRESH]

print(f"Numeryczne: {len(num_all)} (skośne: {len(num_skew)}, normalne+stable: {len(num_normal)})")
print(f"Kat. niska kard.: {len(cat_low)} -> {cat_low}")
print(f"Kat. wysoka kard.: {len(cat_high)} -> {cat_high}")

# 4. ColumnTransformer
def signed_log1p(x):
    # Kompresuje skośne rozkłady; działa dla x ujemnych (w odróżnieniu od log1p)
    return np.sign(x) * np.log1p(np.abs(x))

preprocessor = ColumnTransformer(transformers=[
    ('num_skew', Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('log', FunctionTransformer(signed_log1p, validate=False)),
        ('scl', MinMaxScaler()),
    ]), num_skew),
    ('num_normal', Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('scl', MinMaxScaler()),
    ]), num_normal),
    ('cat_low', Pipeline([
        ('imp', SimpleImputer(strategy='constant', fill_value='Unknown')),
        ('ohe', OneHotEncoder(handle_unknown='ignore', sparse_output=False, drop=False)),
    ]), cat_low),
    ('cat_high', Pipeline([

```

```

        ('imp', SimpleImputer(strategy='constant', fill_value='Unknown')),
        ('enc', TargetEncoder(target_type='binary', smooth='auto')), # kategori
        ('scl', MinMaxScaler()),
    ]), cat_high),
], remainder='drop')

# 5. Fit & Transform (fit tylko na train)
preprocessor.fit(X_train, y_train)

X_train_proc = preprocessor.transform(X_train).astype(np.float32)
X_valid_proc = preprocessor.transform(X_valid).astype(np.float32)
X_test_proc = preprocessor.transform(X_test).astype(np.float32)

assert not np.isnan(X_train_proc).any(), "NaN w X_train_proc!"
assert not np.isnan(X_valid_proc).any(), "NaN w X_valid_proc!"
assert not np.isnan(X_test_proc).any(), "NaN w X_test_proc!"

print(f"\nGotowe macierze Numpy:")
print(f"X_train_proc : {X_train_proc.shape}")
print(f"X_valid_proc : {X_valid_proc.shape}")
print(f"X_test_proc : {X_test_proc.shape}")
print(f"dtype : {X_train_proc.dtype}")

# 7. Zapis
joblib.dump(preprocessor, 'preprocessor.pkl')
joblib.dump(X_train.columns.tolist(), 'input_cols.pkl') # finalna lista kolumn
print("\nZapisano: preprocessor.pkl, input_cols.pkl")

```

```

c:\Users\matma\Documents\SGH\SEM II\Data mining\Projekt_DM\.venv\Lib\site-package
s\numpy\lib\_function_base_impl.py:3023: RuntimeWarning: invalid value encountere
d in divide
  c /= stddev[:, None]
c:\Users\matma\Documents\SGH\SEM II\Data mining\Projekt_DM\.venv\Lib\site-package
s\numpy\lib\_function_base_impl.py:3024: RuntimeWarning: invalid value encountere
d in divide
  c /= stddev[None, :]

```

Kolumny po filtracji: 444 (V_xxx: 321, inne: 68, _isnan: 55)

Usunięto 25 cech o niskiej wariancji -> 402 cech num

Numeryczne: 402 (skośne: 247, normalne+stabe: 155)

Kat. niska kard.: 13 -> ['ProductCD', 'card4', 'card6', 'id_12', 'id_15', 'id_16', 'id_28', 'id_29', 'id_35', 'id_36', 'id_37', 'id_38', 'DeviceType']

Kat. wysoka kard.: 4 -> ['P_emaildomain', 'R_emaildomain', 'id_31', 'DeviceInfo']

Gotowe macierze Numpy:

X_train_proc : (354324, 451)

X_valid_proc : (118108, 451)

X_test_proc : (118108, 451)

dtype : float32

Zapisano: preprocessor.pkl, input_cols.pkl

Trenowanie różnie sparametryzowanych sieci

import potrzebnych bibliotek i stworzenie folderów na wyniki eksperymentów

```
In [4]: import os
import json
import time
import random
from copy import deepcopy
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
from sklearn.metrics import (
    f1_score, precision_score, recall_score,
    roc_auc_score, average_precision_score,
    precision_recall_curve,
)
import matplotlib.pyplot as plt

SEED = 123
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {DEVICE}")

# Tworzenie katalogów na wyniki eksperymentów
ART_DIR = Path("artifacts_mlp"); ART_DIR.mkdir(exist_ok=True)
HIST_DIR = ART_DIR / "history"; HIST_DIR.mkdir(exist_ok=True)
PLOT_DIR = ART_DIR / "plots"; PLOT_DIR.mkdir(exist_ok=True)
```

Device: cuda

Definiowanie różnych funkcji straty, których użyjemy w eksperymentach

```
In [5]: class FocalLoss(nn.Module):
    """Focal Loss dla klasyfikacji binarnej (logits -> [0,1] sigmoid w środku).

    L = -alpha_t * (1 - p_t)^gamma * log(p_t)
    alpha to waga klasy pozytywnej; (1-alpha) dla negatywnej.
    """
    def __init__(self, alpha: float = 0.75, gamma: float = 2.0, reduction: str =
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, logits, targets):
        # numerycznie stabilne BCE per-element
        bce = nn.functional.binary_cross_entropy_with_logits(
            logits, targets, reduction="none"
        )
        p = torch.sigmoid(logits)
        p_t = p * targets + (1 - p) * (1 - targets)
        alpha_t = self.alpha * targets + (1 - self.alpha) * (1 - targets)
        loss = alpha_t * (1 - p_t) ** self.gamma * bce
        if self.reduction == "mean":
            return loss.mean()
```

```

        if self.reduction == "sum":
            return loss.sum()
        return loss

class DiceLoss(nn.Module):
    """Soft Dice Loss dla klasyfikacji binarnej.

    L = 1 - 2*sum(p*y) / (sum(p) + sum(y) + eps)
    Działa na probach po sigmoid; niezbalansowanie nie wpływa,
    bo licznik i mianownik skalują się tak samo.
    """
    def __init__(self, eps: float = 1.0):
        super().__init__()
        self.eps = eps

    def forward(self, logits, targets):
        p = torch.sigmoid(logits)
        inter = (p * targets).sum()
        denom = p.sum() + targets.sum() + self.eps
        return 1.0 - (2.0 * inter + self.eps) / denom

def build_loss(loss_name: str, pos_weight: torch.Tensor | None):
    if loss_name == "BCEWithLogitsLoss+pos_weight":
        return nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    if loss_name == "FocalLoss(alpha=0.75,gamma=2.0)":
        return FocalLoss(alpha=0.75, gamma=2.0)
    if loss_name == "DiceLoss":
        return DiceLoss()
    raise ValueError(loss_name)

```

Definiowanie badanych przez nas architektur modeli na potrzeby funkcji treningowej (potem w kodzie)

```

In [6]: class SimpleMLP(nn.Module):
        """451 -> 128 -> 1. Jedna warstwa ukryta."""
        def __init__(self, in_dim: int, hidden: int = 128, dropout: float = 0.15):
            super().__init__()
            self.net = nn.Sequential(
                nn.Linear(in_dim, hidden),
                nn.ReLU(),
                nn.Dropout(dropout),
                nn.Linear(hidden, 1),
            )

        def forward(self, x):
            return self.net(x).squeeze(-1)

class MediumMLP(nn.Module):
    """451 -> 256 -> 128 -> 64 -> 1 (LayerNorm + Dropout=0.30). Baseline z run_i
    def __init__(self, in_dim: int, dropout: float = 0.30):
        super().__init__()
        dims = [in_dim, 256, 128, 64]
        layers = []
        for i in range(len(dims) - 1):
            layers += [

```

```

        nn.Linear(dims[i], dims[i + 1]),
        nn.LayerNorm(dims[i + 1]),
        nn.ReLU(),
        nn.Dropout(dropout),
    ]
    layers.append(nn.Linear(dims[-1], 1))
    self.net = nn.Sequential(*layers)

def forward(self, x):
    return self.net(x).squeeze(-1)

class ResidualBlock(nn.Module):
    """LayerNorm + Linear + LeakyReLU + Dropout + Linear, ze skip connection.
    Jeśli wymiary się nie zgadzają - liniowa projekcja w skip."""
    def __init__(self, in_dim: int, out_dim: int, dropout: float = 0.15,
                 negative_slope: float = 0.01):
        super().__init__()
        self.norm = nn.LayerNorm(in_dim)
        self.fc1 = nn.Linear(in_dim, out_dim)
        self.act = nn.LeakyReLU(negative_slope)
        self.drop = nn.Dropout(dropout)
        self.fc2 = nn.Linear(out_dim, out_dim)
        self.skip = nn.Linear(in_dim, out_dim) if in_dim != out_dim else nn.Identity

    def forward(self, x):
        h = self.norm(x)
        h = self.fc1(h)
        h = self.act(h)
        h = self.drop(h)
        h = self.fc2(h)
        return self.act(h + self.skip(x))

class AdvancedResMLP(nn.Module):
    """451 -> 512 -> 256 -> 128 -> 64 -> 1, z residual blockami + LeakyReLU."""
    def __init__(self, in_dim: int, dropout: float = 0.15):
        super().__init__()
        self.blocks = nn.Sequential(
            ResidualBlock(in_dim, 512, dropout),
            ResidualBlock(512, 256, dropout),
            ResidualBlock(256, 128, dropout),
            ResidualBlock(128, 64, dropout),
        )
        self.head = nn.Linear(64, 1)

    def forward(self, x):
        return self.head(self.blocks(x)).squeeze(-1)

# Domyślny dropout per architektura - odwzorowuje wartości używane w grid search
# build_model(..., dropout=None) => użyj tych wartości (zachowuje stare wyniki).
DEFAULT_DROPOUT = {"simple": 0.15, "medium": 0.15, "advanced": 0.15}

def build_model(arch: str, in_dim: int, dropout: float | None = None) -> nn.Module:
    # dropout=None -> domyślny dla danej architektury (jak w grid searchu).
    # Podaj wartość jawnie, gdy chcesz ją nadpisać (np. tuning runów 10-12),
    # zamiast edytować definicję modelu.
    d = DEFAULT_DROPOUT[arch] if dropout is None else dropout

```

```

if arch == "simple":
    return SimpleMLP(in_dim, dropout=d)
if arch == "medium":
    return MediumMLP(in_dim, dropout=d)
if arch == "advanced":
    return AdvancedResMLP(in_dim, dropout=d)
raise ValueError(arch)

```

Definicje Early Stopping oraz funkcji do ewaluacji modeli

```

In [7]: class EarlyStopping:
        """Maksymalizacja metryki na valid (PR-AUC lub F1). Przechowuje best_state_d
        def __init__(self, patience: int = 10, mode: str = "max"):
            self.patience = patience
            self.mode = mode
            self.best = -np.inf if mode == "max" else np.inf
            self.counter = 0
            self.best_state = None
            self.best_epoch = -1

        def step(self, score: float, model: nn.Module, epoch: int) -> bool:
            improved = (self.mode == "max" and score > self.best) or \
                (self.mode == "min" and score < self.best)
            if improved:
                self.best = score
                self.best_state = deepcopy(model.state_dict())
                self.best_epoch = epoch
                self.counter = 0
                return False
            self.counter += 1
            return self.counter >= self.patience

        @torch.no_grad()
        def predict_proba(model: nn.Module, X: np.ndarray, batch_size: int = 4096) -> np:
            model.eval()
            out = []
            for i in range(0, len(X), batch_size):
                xb = torch.from_numpy(X[i:i + batch_size]).to(DEVICE)
                out.append(torch.sigmoid(model(xb)).cpu().numpy())
            return np.concatenate(out)

        def best_f1_threshold(y_true: np.ndarray, y_proba: np.ndarray) -> tuple[float, f
            """Próg maksymalizujący F1 (przez precision_recall_curve - tańsze niż grid).
            p, r, t = precision_recall_curve(y_true, y_proba)
            # f1 dla każdego punktu (ostatni punkt nie ma odpowiadającego thresholdu)
            f1 = 2 * p * r / (p + r + 1e-12)
            f1 = f1[:-1] # przycinamy do długości t
            if len(f1) == 0:
                return 0.5, 0.0
            idx = int(np.argmax(f1))
            return float(t[idx]), float(f1[idx])

        def evaluate(y_true: np.ndarray, y_proba: np.ndarray, threshold: float) -> dict:
            y_pred = (y_proba >= threshold).astype(int)
            return {
                "F1": f1_score(y_true, y_pred, zero_division=0),

```



```

    "Precision": precision_score(y_true, y_pred, zero_division=0),
    "Recall":    recall_score(y_true, y_pred, zero_division=0),
    "ROC_AUC":   roc_auc_score(y_true, y_proba),
    "AUPRC":     average_precision_score(y_true, y_proba),
}

```

Zdefiniowanie pętli treningowej

```

In [8]: def train_one(
    model: nn.Module,
    X_train: np.ndarray, y_train: np.ndarray,
    X_valid: np.ndarray, y_valid: np.ndarray,
    loss_fn: nn.Module,
    *,
    max_epochs: int = 80,
    batch_size: int = 2048,
    lr: float = 1e-3,
    weight_decay: float = 1e-5,
    patience: int = 10,
    es_metric: str = "AUPRC", # "AUPRC" lub "F1"
    verbose: bool = True,
) -> tuple[dict, dict]:
    """Zwraca (history, best_info). best_info ma best_state_dict, best_epoch, be
ds = TensorDataset(torch.from_numpy(X_train), torch.from_numpy(y_train.astype
dl = DataLoader(ds, batch_size=batch_size, shuffle=True, num_workers=0, drop

    model = model.to(DEVICE)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        opt, mode="min", factor=0.5, patience=5, min_lr=1e-6
    )
    es = EarlyStopping(patience=patience, mode="max")

    history = {
        "epoch": [], "train_loss": [], "val_loss": [],
        "val_F1": [], "val_Precision": [], "val_Recall": [],
        "val_ROC_AUC": [], "val_AUPRC": [], "val_threshold": [],
    }

    for epoch in range(1, max_epochs + 1):
        # ---- trening ----
        model.train()
        tot_loss, n_seen = 0.0, 0
        for xb, yb in dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            logits = model(xb)
            loss = loss_fn(logits, yb)
            loss.backward()
            opt.step()
            tot_loss += loss.item() * xb.size(0); n_seen += xb.size(0)
        train_loss = tot_loss / n_seen

        # ---- walidacja ----
        with torch.no_grad():
            model.eval()
            val_logits = []
            for i in range(0, len(X_valid), batch_size):
                xb = torch.from_numpy(X_valid[i:i + batch_size]).to(DEVICE)

```

```

        val_logits.append(model(xb))
    val_logits = torch.cat(val_logits) # GPU
    val_targets = torch.from_numpy(y_valid.astype(np.float32)).to(DEVICE)
    val_loss = float(loss_fn(val_logits, val_targets).item())
    val_proba = torch.sigmoid(val_logits).cpu().numpy()

    scheduler.step(val_loss)

    thr, _ = best_f1_threshold(y_valid, val_proba)
    m = evaluate(y_valid, val_proba, thr)

    history["epoch"].append(epoch)
    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["val_F1"].append(m["F1"])
    history["val_Precision"].append(m["Precision"])
    history["val_Recall"].append(m["Recall"])
    history["val_ROC_AUC"].append(m["ROC_AUC"])
    history["val_AUPRC"].append(m["AUPRC"])
    history["val_threshold"].append(thr)

    cur_lr = opt.param_groups[0]["lr"]
    score = m[es_metric]
    stop = es.step(score, model, epoch)
    if verbose:
        print(f" ep{epoch:03d} | tr_loss={train_loss:.4f} val_loss={val_loss:.4f} F1={m['F1']:.4f} AUPRC={m['AUPRC']:.4f} ROC_AUC={m['ROC_AUC']:.4f} thr={thr:.3f} lr={cur_lr:.2e} | best_{es_metric}={es.best:.4f}")
    if stop:
        if verbose:
            print(f" ■ early stop @ ep{epoch} (no improv {patience} epochs)
        break

    # przywróć najlepsze wagi
    if es.best_state is not None:
        model.load_state_dict(es.best_state)
    return history, {"best_epoch": es.best_epoch, "best_score": es.best, "es_metric": es.metric}

```

Zdefiniowanie funkcji do stworzenia wykresu z przebiegu treningu

```

In [9]: def plot_history(history: dict, title: str, save_path: Path | None = None,
                        best_epoch: int | None = None):
    fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

    ep = history["epoch"]

    # wygładzone val_loss (rolling mean, okno=5) - czytelniejsze przy pos_weight
    val_loss_raw = np.array(history["val_loss"])
    kernel = min(5, len(val_loss_raw))
    val_loss_smooth = np.convolve(val_loss_raw, np.ones(kernel) / kernel, mode="same")

    axes[0].plot(ep, history["train_loss"], label="train_loss")
    axes[0].plot(ep, val_loss_raw, alpha=0.25, color="tab:orange", label="val_loss_raw")
    axes[0].plot(ep, val_loss_smooth, color="tab:orange", label="val_loss_smooth")
    axes[0].set_xlabel("epoch"); axes[0].set_ylabel("loss")
    axes[0].set_title("Loss"); axes[0].legend(); axes[0].grid(alpha=0.3)

```

```

axes[1].plot(ep, history["val_AUPRC"], label="AUPRC (ES metric)")
axes[1].plot(ep, history["val_ROC_AUC"], label="ROC_AUC")
axes[1].plot(ep, history["val_F1"], label="F1", linestyle="--", alpha=0.3)
axes[1].set_xlabel("epoch"); axes[1].set_ylabel("score")
axes[1].set_title("Valid metrics"); axes[1].legend(); axes[1].grid(alpha=0.3)

if best_epoch is not None:
    for ax in axes:
        ax.axvline(best_epoch, color="red", linestyle=":", linewidth=1.5,
                    label=f"best ep={best_epoch}")
    axes[0].legend(); axes[1].legend()

fig.suptitle(title)
fig.tight_layout()
if save_path is not None:
    fig.savefig(save_path, dpi=120, bbox_inches="tight")
plt.show()

```

Główna funkcja do treningu i grid searchu parametrów

```

In [10]: RESULTS_XLSX = Path("experiment_results_mlp.xlsx")
RESULTS_COLS = [
    "run_id", "model", "bottleneck", "dropout", "loss_fn",
    "epochs_trained", "val_loss", "scoring_method", "threshold_strategy",
    "F1", "Precision", "Recall", "ROC_AUC", "AUPRC",
    "threshold", "notes",
]

def append_results(row: dict):
    if RESULTS_XLSX.exists():
        df = pd.read_excel(RESULTS_XLSX)
    else:
        df = pd.DataFrame(columns=RESULTS_COLS)
    df = pd.concat([df, pd.DataFrame([row], columns=RESULTS_COLS)], ignore_index=True)
    df.to_excel(RESULTS_XLSX, index=False)

def next_run_id() -> int:
    if not RESULTS_XLSX.exists():
        return 1
    df = pd.read_excel(RESULTS_XLSX)
    return int(df["run_id"].max()) + 1 if len(df) else 1

def run_grid_search(
    X_train_proc, y_train, X_valid_proc, y_valid, X_test_proc, y_test,
    architectures=("simple", "medium", "advanced"),
    losses=("BCEWithLogitsLoss+pos_weight",
            "FocalLoss(alpha=0.75,gamma=2.0)",
            "DiceLoss"),
    max_epochs: int = 100,
    patience: int = 10,
    es_metric: str = "AUPRC",
    pos_weight_cap: float | None = 10.0,
    lr: float = 1e-3,
    batch_size: int = 2048,
    dropout: float | None = None, # NOWE: None = domyślny per arch (stare wyniki)
    tag_suffix: str = "",         # NOWE: dopisek do nazw plików, by tuning ni

```

```

):
    """Iteruje po (arch x loss). Zapisuje best wagi, historię JSON i wiersz w xl
    # konwersje
    Xtr = X_train_proc.astype(np.float32)
    Xva = X_valid_proc.astype(np.float32)
    Xte = X_test_proc.astype(np.float32)
    ytr = np.asarray(y_train).astype(np.float32)
    yva = np.asarray(y_valid).astype(np.float32)
    yte = np.asarray(y_test).astype(np.float32)

    in_dim = Xtr.shape[1]

    # pos_weight - proporcja neg/pos w treningu (z opcjonalnym capem)
    pos = float((ytr == 1).sum()); neg = float((ytr == 0).sum())
    pw = neg / max(pos, 1.0)
    pw_used = pw if pos_weight_cap is None else min(pw, pos_weight_cap)
    pos_weight_tensor = torch.tensor([pw_used], dtype=torch.float32, device=DEVI
    print(f"pos_weight raw={pw:.2f}, used={pw_used:.2f} "
          f"(cap={pos_weight_cap}); train_pos_rate={pos/(pos+neg):.4f}")

    summary_rows = []

    for arch in architectures:
        for loss_name in losses:
            tag = f"{arch}_{loss_name}".replace(" ", "").replace("/", "")
            tag_safe = (tag
                        .replace("+", "_")
                        .replace("(", "_").replace(")", "")
                        .replace("=", "").replace(",", "_").replace(".", ""))
            tag_safe = tag_safe + tag_suffix # NOWE: dopisek chroni przed nadp
            # NOWE: faktycznie użyty dropout (do build_model i do zapisu w xlsx)
            dropout_used = DEFAULT_DROPOUT[arch] if dropout is None else dropout
            print(f"\n===== {arch.upper()} | {loss_name} "
                  f"| dropout={dropout_used} =====")
            torch.manual_seed(SEED); np.random.seed(SEED)

            model = build_model(arch, in_dim, dropout=dropout) # NOWE: przekaz
            loss_fn = build_loss(loss_name, pos_weight_tensor)

            t0 = time.time()
            history, best = train_one(
                model, Xtr, ytr, Xva, yva, loss_fn,
                max_epochs=max_epochs, patience=patience,
                es_metric=es_metric, verbose=True,
                lr=lr, batch_size=batch_size,
            )
            elapsed = time.time() - t0

            # zapis wag
            weights_path = ART_DIR / f"best_mlp_{tag_safe}.pth"
            torch.save(model.state_dict(), weights_path)

            # zapis historii
            hist_path = HIST_DIR / f"history_{tag_safe}.json"
            with open(hist_path, "w") as f:
                json.dump(history, f, indent=2)

            # wykres
            plot_history(history, title=f"{arch} | {loss_name}",
                        save_path=PLOT_DIR / f"loss_{tag_safe}.png",

```

```

        best_epoch=best["best_epoch"])

# ewaluacja: progi z valid -> test
val_proba = predict_proba(model, Xva)
thr, _ = best_f1_threshold(yva, val_proba)
val_m = evaluate(yva, val_proba, thr)

test_proba = predict_proba(model, Xte)
test_m = evaluate(yte, test_proba, thr)

# idx najlepszej epoki w history (do val_loss)
best_ep_idx = history["epoch"].index(best["best_epoch"])
val_loss_at_best = history["val_loss"][best_ep_idx]
epochs_trained = len(history["epoch"])

notes = (
    f"arch={arch} | loss={loss_name} | "
    f"pos_w_used={pw_used:.2f} | lr={lr} wd=1e-5 bs={batch_size} "
    f"patience={patience} ES={es_metric} | "
    f"best_ep={best['best_epoch']} time={elapsed:.0f}s | "
    f"test_F1={test_m['F1']:.4f} test_AUC={test_m['ROC_AUC']:.4f} "
    f"test_Gini={2*test_m['ROC_AUC']-1:.4f} test_AUPRC={test_m['AUPRC']:.4f} "
)

row = {
    "run_id": next_run_id(),
    "model": f"FraudMLP_{arch}",
    "bottleneck": "-",
    "dropout": round(dropout_used, 4), # NOWE: faktyczny
    "loss_fn": loss_name,
    "epochs_trained": epochs_trained,
    "val_loss": round(val_loss_at_best, 6),
    "scoring_method": "sigmoid_prob",
    "threshold_strategy": "max_F1_on_valid",
    "F1": round(val_m["F1"], 4),
    "Precision": round(val_m["Precision"], 4),
    "Recall": round(val_m["Recall"], 4),
    "ROC_AUC": round(val_m["ROC_AUC"], 4),
    "AUPRC": round(val_m["AUPRC"], 4),
    "threshold": round(thr, 4),
    "notes": notes,
}

append_results(row)
summary_rows.append(**row,
                    "test_F1": test_m["F1"],
                    "test_AUC": test_m["ROC_AUC"],
                    "test_AUPRC": test_m["AUPRC"])

print(f" weights -> {weights_path.name}")
print(f" history -> {hist_path.name}")
print(f" VAL : {val_m}")
print(f" TEST : {test_m} (thr={thr:.4f})")

return pd.DataFrame(summary_rows)

```

Wywołanie kodu

```

In [11]: summary = run_grid_search(
        X_train_proc, y_train, X_valid_proc, y_valid, X_test_proc, y_test,

```

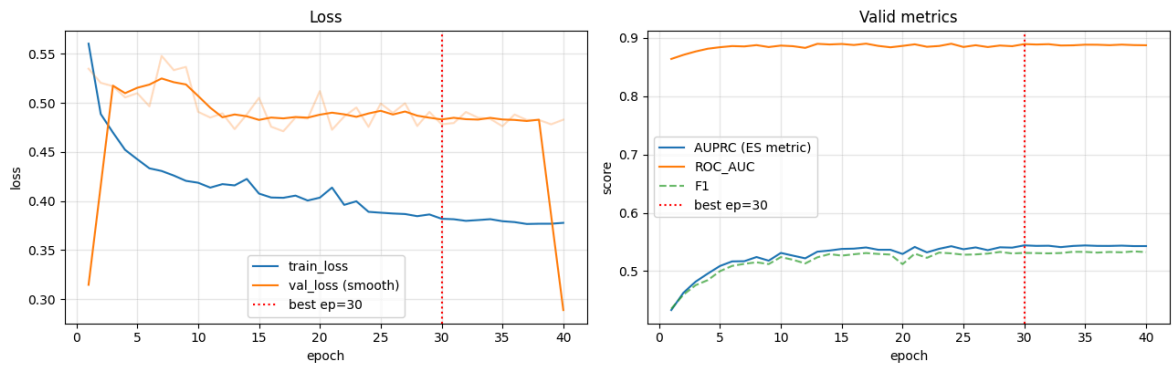
```
architectures=("simple", "medium", "advanced"),
losses=("BCEWithLogitsLoss+pos_weight",
        "FocalLoss(alpha=0.75,gamma=2.0)",
        "DiceLoss"),
max_epochs=80,
patience=10,
es_metric="AUPRC",      # albo "F1"
pos_weight_cap=10.0,    # None = bez capa
batch_size=2048,
)
summary.sort_values("AUPRC", ascending=False)
```

pos_weight raw=28.56, used=10.00 (cap=10.0); train_pos_rate=0.0338

```
===== SIMPLE | BCEWithLogitsLoss+pos_weight | dropout=0.15 =====
ep001 | tr_loss=0.5605 val_loss=0.5350 F1=0.4355 AUPRC=0.4333 ROC_AUC=0.8637 th
r=0.613 lr=1.00e-03 | best_AUPRC=0.4333 (ep1)
ep002 | tr_loss=0.4887 val_loss=0.5205 F1=0.4597 AUPRC=0.4630 ROC_AUC=0.8707 th
r=0.720 lr=1.00e-03 | best_AUPRC=0.4630 (ep2)
ep003 | tr_loss=0.4699 val_loss=0.5172 F1=0.4760 AUPRC=0.4820 ROC_AUC=0.8765 th
r=0.561 lr=1.00e-03 | best_AUPRC=0.4820 (ep3)
ep004 | tr_loss=0.4522 val_loss=0.5057 F1=0.4846 AUPRC=0.4958 ROC_AUC=0.8813 th
r=0.693 lr=1.00e-03 | best_AUPRC=0.4958 (ep4)
ep005 | tr_loss=0.4426 val_loss=0.5100 F1=0.5000 AUPRC=0.5089 ROC_AUC=0.8838 th
r=0.635 lr=1.00e-03 | best_AUPRC=0.5089 (ep5)
ep006 | tr_loss=0.4333 val_loss=0.4966 F1=0.5090 AUPRC=0.5167 ROC_AUC=0.8858 th
r=0.581 lr=1.00e-03 | best_AUPRC=0.5167 (ep6)
ep007 | tr_loss=0.4306 val_loss=0.5481 F1=0.5125 AUPRC=0.5170 ROC_AUC=0.8852 th
r=0.492 lr=1.00e-03 | best_AUPRC=0.5170 (ep7)
ep008 | tr_loss=0.4260 val_loss=0.5335 F1=0.5152 AUPRC=0.5242 ROC_AUC=0.8874 th
r=0.571 lr=1.00e-03 | best_AUPRC=0.5242 (ep8)
ep009 | tr_loss=0.4206 val_loss=0.5368 F1=0.5121 AUPRC=0.5179 ROC_AUC=0.8841 th
r=0.495 lr=1.00e-03 | best_AUPRC=0.5242 (ep8)
ep010 | tr_loss=0.4187 val_loss=0.4910 F1=0.5241 AUPRC=0.5312 ROC_AUC=0.8866 th
r=0.630 lr=1.00e-03 | best_AUPRC=0.5312 (ep10)
ep011 | tr_loss=0.4137 val_loss=0.4851 F1=0.5194 AUPRC=0.5266 ROC_AUC=0.8856 th
r=0.766 lr=1.00e-03 | best_AUPRC=0.5312 (ep10)
ep012 | tr_loss=0.4172 val_loss=0.4894 F1=0.5132 AUPRC=0.5220 ROC_AUC=0.8827 th
r=0.777 lr=1.00e-03 | best_AUPRC=0.5312 (ep10)
ep013 | tr_loss=0.4160 val_loss=0.4733 F1=0.5237 AUPRC=0.5334 ROC_AUC=0.8896 th
r=0.730 lr=1.00e-03 | best_AUPRC=0.5334 (ep13)
ep014 | tr_loss=0.4225 val_loss=0.4883 F1=0.5292 AUPRC=0.5353 ROC_AUC=0.8885 th
r=0.661 lr=1.00e-03 | best_AUPRC=0.5353 (ep14)
ep015 | tr_loss=0.4076 val_loss=0.5052 F1=0.5266 AUPRC=0.5381 ROC_AUC=0.8894 th
r=0.603 lr=1.00e-03 | best_AUPRC=0.5381 (ep15)
ep016 | tr_loss=0.4036 val_loss=0.4758 F1=0.5289 AUPRC=0.5386 ROC_AUC=0.8876 th
r=0.705 lr=1.00e-03 | best_AUPRC=0.5386 (ep16)
ep017 | tr_loss=0.4033 val_loss=0.4712 F1=0.5310 AUPRC=0.5406 ROC_AUC=0.8899 th
r=0.760 lr=1.00e-03 | best_AUPRC=0.5406 (ep17)
ep018 | tr_loss=0.4055 val_loss=0.4852 F1=0.5295 AUPRC=0.5366 ROC_AUC=0.8861 th
r=0.672 lr=1.00e-03 | best_AUPRC=0.5406 (ep17)
ep019 | tr_loss=0.4005 val_loss=0.4841 F1=0.5284 AUPRC=0.5366 ROC_AUC=0.8837 th
r=0.691 lr=1.00e-03 | best_AUPRC=0.5406 (ep17)
ep020 | tr_loss=0.4034 val_loss=0.5122 F1=0.5120 AUPRC=0.5294 ROC_AUC=0.8861 th
r=0.882 lr=1.00e-03 | best_AUPRC=0.5406 (ep17)
ep021 | tr_loss=0.4137 val_loss=0.4727 F1=0.5300 AUPRC=0.5415 ROC_AUC=0.8888 th
r=0.758 lr=1.00e-03 | best_AUPRC=0.5415 (ep21)
ep022 | tr_loss=0.3961 val_loss=0.4862 F1=0.5228 AUPRC=0.5322 ROC_AUC=0.8846 th
r=0.794 lr=1.00e-03 | best_AUPRC=0.5415 (ep21)
ep023 | tr_loss=0.3997 val_loss=0.4954 F1=0.5318 AUPRC=0.5383 ROC_AUC=0.8859 th
r=0.630 lr=5.00e-04 | best_AUPRC=0.5415 (ep21)
ep024 | tr_loss=0.3890 val_loss=0.4757 F1=0.5305 AUPRC=0.5427 ROC_AUC=0.8898 th
r=0.751 lr=5.00e-04 | best_AUPRC=0.5427 (ep24)
ep025 | tr_loss=0.3881 val_loss=0.4995 F1=0.5281 AUPRC=0.5375 ROC_AUC=0.8843 th
r=0.656 lr=5.00e-04 | best_AUPRC=0.5427 (ep24)
ep026 | tr_loss=0.3873 val_loss=0.4899 F1=0.5287 AUPRC=0.5407 ROC_AUC=0.8872 th
r=0.669 lr=5.00e-04 | best_AUPRC=0.5427 (ep24)
ep027 | tr_loss=0.3867 val_loss=0.4997 F1=0.5301 AUPRC=0.5359 ROC_AUC=0.8841 th
r=0.666 lr=5.00e-04 | best_AUPRC=0.5427 (ep24)
ep028 | tr_loss=0.3846 val_loss=0.4765 F1=0.5329 AUPRC=0.5409 ROC_AUC=0.8867 th
r=0.711 lr=5.00e-04 | best_AUPRC=0.5427 (ep24)
ep029 | tr_loss=0.3863 val_loss=0.4909 F1=0.5303 AUPRC=0.5403 ROC_AUC=0.8856 th
```

r=0.646 lr=2.50e-04 | best_AUPRC=0.5427 (ep24)
 ep030 | tr_loss=0.3819 val_loss=0.4783 F1=0.5312 AUPRC=0.5442 ROC_AUC=0.8891 th
 r=0.702 lr=2.50e-04 | best_AUPRC=0.5442 (ep30)
 ep031 | tr_loss=0.3814 val_loss=0.4795 F1=0.5309 AUPRC=0.5433 ROC_AUC=0.8885 th
 r=0.687 lr=2.50e-04 | best_AUPRC=0.5442 (ep30)
 ep032 | tr_loss=0.3798 val_loss=0.4908 F1=0.5305 AUPRC=0.5435 ROC_AUC=0.8890 th
 r=0.699 lr=2.50e-04 | best_AUPRC=0.5442 (ep30)
 ep033 | tr_loss=0.3806 val_loss=0.4850 F1=0.5310 AUPRC=0.5412 ROC_AUC=0.8868 th
 r=0.667 lr=2.50e-04 | best_AUPRC=0.5442 (ep30)
 ep034 | tr_loss=0.3815 val_loss=0.4838 F1=0.5332 AUPRC=0.5432 ROC_AUC=0.8871 th
 r=0.689 lr=2.50e-04 | best_AUPRC=0.5442 (ep30)
 ep035 | tr_loss=0.3794 val_loss=0.4763 F1=0.5330 AUPRC=0.5441 ROC_AUC=0.8882 th
 r=0.712 lr=1.25e-04 | best_AUPRC=0.5442 (ep30)
 ep036 | tr_loss=0.3785 val_loss=0.4882 F1=0.5318 AUPRC=0.5433 ROC_AUC=0.8881 th
 r=0.709 lr=1.25e-04 | best_AUPRC=0.5442 (ep30)
 ep037 | tr_loss=0.3766 val_loss=0.4826 F1=0.5329 AUPRC=0.5432 ROC_AUC=0.8873 th
 r=0.676 lr=1.25e-04 | best_AUPRC=0.5442 (ep30)
 ep038 | tr_loss=0.3768 val_loss=0.4827 F1=0.5323 AUPRC=0.5437 ROC_AUC=0.8883 th
 r=0.724 lr=1.25e-04 | best_AUPRC=0.5442 (ep30)
 ep039 | tr_loss=0.3768 val_loss=0.4782 F1=0.5339 AUPRC=0.5429 ROC_AUC=0.8875 th
 r=0.742 lr=1.25e-04 | best_AUPRC=0.5442 (ep30)
 ep040 | tr_loss=0.3777 val_loss=0.4830 F1=0.5326 AUPRC=0.5429 ROC_AUC=0.8872 th
 r=0.686 lr=1.25e-04 | best_AUPRC=0.5442 (ep30)
 ■ early stop @ ep40 (no improv 10 epochs)

simple | BCEWithLogitsLoss+pos_weight

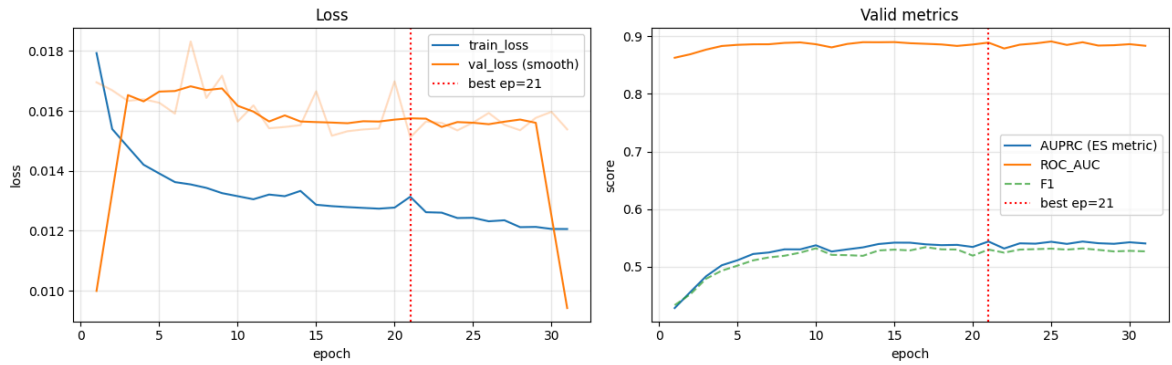



```
weights -> best_mlp_simple_BCEWithLogitsLoss_pos_weight.pth
history -> history_simple_BCEWithLogitsLoss_pos_weight.json
VAL : {'F1': 0.53125, 'Precision': 0.6076514939960905, 'Recall': 0.47191498590
32748, 'ROC_AUC': 0.8891489162517109, 'AUPRC': 0.5441547520038034}
TEST : {'F1': 0.4461559248750169, 'Precision': 0.49475576865448007, 'Recall':
0.40625, 'ROC_AUC': 0.8599858120446398, 'AUPRC': 0.397719554468316} (thr=0.7020)
```

```
===== SIMPLE | FocalLoss(alpha=0.75,gamma=2.0) | dropout=0.15 =====
ep001 | tr_loss=0.0179 val_loss=0.0170 F1=0.4336 AUPRC=0.4286 ROC_AUC=0.8625 th
r=0.453 lr=1.00e-03 | best_AUPRC=0.4286 (ep1)
ep002 | tr_loss=0.0154 val_loss=0.0167 F1=0.4526 AUPRC=0.4564 ROC_AUC=0.8685 th
r=0.484 lr=1.00e-03 | best_AUPRC=0.4564 (ep2)
ep003 | tr_loss=0.0148 val_loss=0.0163 F1=0.4794 AUPRC=0.4836 ROC_AUC=0.8765 th
r=0.425 lr=1.00e-03 | best_AUPRC=0.4836 (ep3)
ep004 | tr_loss=0.0142 val_loss=0.0164 F1=0.4934 AUPRC=0.5027 ROC_AUC=0.8828 th
r=0.418 lr=1.00e-03 | best_AUPRC=0.5027 (ep4)
ep005 | tr_loss=0.0139 val_loss=0.0163 F1=0.5021 AUPRC=0.5114 ROC_AUC=0.8848 th
r=0.437 lr=1.00e-03 | best_AUPRC=0.5114 (ep5)
ep006 | tr_loss=0.0136 val_loss=0.0159 F1=0.5112 AUPRC=0.5223 ROC_AUC=0.8858 th
r=0.421 lr=1.00e-03 | best_AUPRC=0.5223 (ep6)
ep007 | tr_loss=0.0135 val_loss=0.0183 F1=0.5164 AUPRC=0.5249 ROC_AUC=0.8858 th
r=0.359 lr=1.00e-03 | best_AUPRC=0.5249 (ep7)
ep008 | tr_loss=0.0134 val_loss=0.0164 F1=0.5192 AUPRC=0.5303 ROC_AUC=0.8882 th
r=0.439 lr=1.00e-03 | best_AUPRC=0.5303 (ep8)
ep009 | tr_loss=0.0133 val_loss=0.0172 F1=0.5246 AUPRC=0.5303 ROC_AUC=0.8890 th
r=0.395 lr=1.00e-03 | best_AUPRC=0.5303 (ep8)
ep010 | tr_loss=0.0132 val_loss=0.0156 F1=0.5322 AUPRC=0.5374 ROC_AUC=0.8859 th
r=0.430 lr=1.00e-03 | best_AUPRC=0.5374 (ep10)
ep011 | tr_loss=0.0131 val_loss=0.0162 F1=0.5207 AUPRC=0.5265 ROC_AUC=0.8804 th
r=0.525 lr=1.00e-03 | best_AUPRC=0.5374 (ep10)
ep012 | tr_loss=0.0132 val_loss=0.0154 F1=0.5203 AUPRC=0.5302 ROC_AUC=0.8864 th
r=0.513 lr=1.00e-03 | best_AUPRC=0.5374 (ep10)
ep013 | tr_loss=0.0132 val_loss=0.0155 F1=0.5191 AUPRC=0.5338 ROC_AUC=0.8894 th
r=0.525 lr=1.00e-03 | best_AUPRC=0.5374 (ep10)
ep014 | tr_loss=0.0133 val_loss=0.0155 F1=0.5284 AUPRC=0.5396 ROC_AUC=0.8893 th
r=0.454 lr=1.00e-03 | best_AUPRC=0.5396 (ep14)
ep015 | tr_loss=0.0129 val_loss=0.0167 F1=0.5298 AUPRC=0.5419 ROC_AUC=0.8896 th
r=0.418 lr=1.00e-03 | best_AUPRC=0.5419 (ep15)
ep016 | tr_loss=0.0128 val_loss=0.0152 F1=0.5284 AUPRC=0.5418 ROC_AUC=0.8876 th
r=0.478 lr=1.00e-03 | best_AUPRC=0.5419 (ep15)
ep017 | tr_loss=0.0128 val_loss=0.0153 F1=0.5341 AUPRC=0.5390 ROC_AUC=0.8866 th
r=0.472 lr=1.00e-03 | best_AUPRC=0.5419 (ep15)
ep018 | tr_loss=0.0128 val_loss=0.0154 F1=0.5303 AUPRC=0.5376 ROC_AUC=0.8855 th
r=0.488 lr=1.00e-03 | best_AUPRC=0.5419 (ep15)
ep019 | tr_loss=0.0127 val_loss=0.0154 F1=0.5300 AUPRC=0.5381 ROC_AUC=0.8829 th
r=0.486 lr=1.00e-03 | best_AUPRC=0.5419 (ep15)
ep020 | tr_loss=0.0128 val_loss=0.0170 F1=0.5194 AUPRC=0.5344 ROC_AUC=0.8854 th
r=0.548 lr=1.00e-03 | best_AUPRC=0.5419 (ep15)
ep021 | tr_loss=0.0131 val_loss=0.0151 F1=0.5299 AUPRC=0.5439 ROC_AUC=0.8887 th
r=0.477 lr=1.00e-03 | best_AUPRC=0.5439 (ep21)
ep022 | tr_loss=0.0126 val_loss=0.0156 F1=0.5246 AUPRC=0.5318 ROC_AUC=0.8785 th
r=0.513 lr=1.00e-03 | best_AUPRC=0.5439 (ep21)
ep023 | tr_loss=0.0126 val_loss=0.0156 F1=0.5299 AUPRC=0.5407 ROC_AUC=0.8851 th
r=0.441 lr=1.00e-03 | best_AUPRC=0.5439 (ep21)
ep024 | tr_loss=0.0124 val_loss=0.0153 F1=0.5307 AUPRC=0.5401 ROC_AUC=0.8873 th
r=0.468 lr=1.00e-03 | best_AUPRC=0.5439 (ep21)
ep025 | tr_loss=0.0124 val_loss=0.0156 F1=0.5316 AUPRC=0.5434 ROC_AUC=0.8907 th
r=0.472 lr=1.00e-03 | best_AUPRC=0.5439 (ep21)
ep026 | tr_loss=0.0123 val_loss=0.0159 F1=0.5299 AUPRC=0.5398 ROC_AUC=0.8847 th
r=0.439 lr=1.00e-03 | best_AUPRC=0.5439 (ep21)
```

ep027 | tr_loss=0.0124 val_loss=0.0155 F1=0.5319 AUPRC=0.5439 ROC_AUC=0.8893 th
 r=0.471 lr=5.00e-04 | best_AUPRC=0.5439 (ep21)
 ep028 | tr_loss=0.0121 val_loss=0.0154 F1=0.5294 AUPRC=0.5409 ROC_AUC=0.8836 th
 r=0.474 lr=5.00e-04 | best_AUPRC=0.5439 (ep21)
 ep029 | tr_loss=0.0121 val_loss=0.0158 F1=0.5267 AUPRC=0.5399 ROC_AUC=0.8842 th
 r=0.461 lr=5.00e-04 | best_AUPRC=0.5439 (ep21)
 ep030 | tr_loss=0.0121 val_loss=0.0160 F1=0.5277 AUPRC=0.5426 ROC_AUC=0.8860 th
 r=0.430 lr=5.00e-04 | best_AUPRC=0.5439 (ep21)
 ep031 | tr_loss=0.0121 val_loss=0.0154 F1=0.5268 AUPRC=0.5405 ROC_AUC=0.8832 th
 r=0.477 lr=5.00e-04 | best_AUPRC=0.5439 (ep21)
 ■ early stop @ ep31 (no improv 10 epochs)

simple | FocalLoss(alpha=0.75,gamma=2.0)



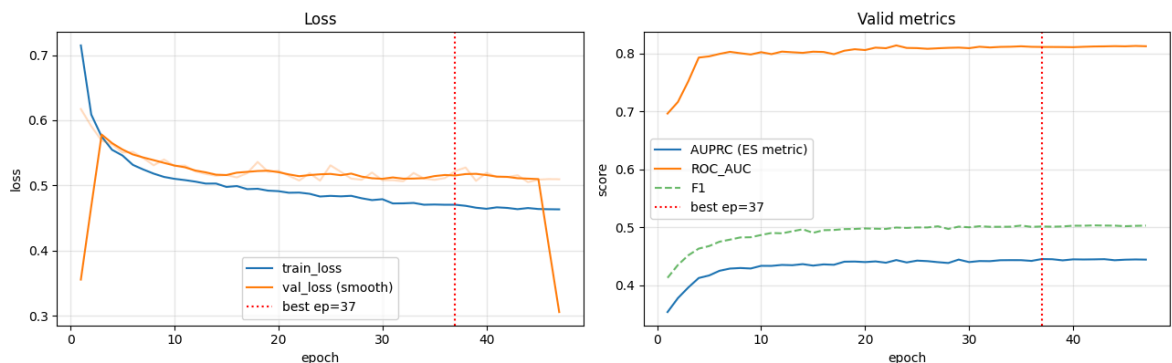
```
weights -> best_mlp_simple_FocalLoss_alpha075_gamma20.pth
history -> history_simple_FocalLoss_alpha075_gamma20.json
VAL : {'F1': 0.5298857486470234, 'Precision': 0.5947624190064795, 'Recall': 0.4777705486879202, 'ROC_AUC': 0.8887036017814581, 'AUPRC': 0.5439090737675105}
TEST : {'F1': 0.4410405387561072, 'Precision': 0.47591906526075806, 'Recall': 0.4109251968503937, 'ROC_AUC': 0.8594500655673165, 'AUPRC': 0.394622767832573} (thr=0.4774)
```

===== SIMPLE | DiceLoss | dropout=0.15 =====

```
ep001 | tr_loss=0.7148 val_loss=0.6172 F1=0.4128 AUPRC=0.3537 ROC_AUC=0.6965 thr=0.691 lr=1.00e-03 | best_AUPRC=0.3537 (ep1)
ep002 | tr_loss=0.6086 val_loss=0.5909 F1=0.4351 AUPRC=0.3778 ROC_AUC=0.7166 thr=0.076 lr=1.00e-03 | best_AUPRC=0.3778 (ep2)
ep003 | tr_loss=0.5748 val_loss=0.5687 F1=0.4517 AUPRC=0.3962 ROC_AUC=0.7525 thr=0.108 lr=1.00e-03 | best_AUPRC=0.3962 (ep3)
ep004 | tr_loss=0.5546 val_loss=0.5628 F1=0.4633 AUPRC=0.4125 ROC_AUC=0.7932 thr=0.104 lr=1.00e-03 | best_AUPRC=0.4125 (ep4)
ep005 | tr_loss=0.5458 val_loss=0.5503 F1=0.4677 AUPRC=0.4169 ROC_AUC=0.7953 thr=0.127 lr=1.00e-03 | best_AUPRC=0.4169 (ep5)
ep006 | tr_loss=0.5315 val_loss=0.5513 F1=0.4752 AUPRC=0.4250 ROC_AUC=0.7996 thr=0.031 lr=1.00e-03 | best_AUPRC=0.4250 (ep6)
ep007 | tr_loss=0.5244 val_loss=0.5419 F1=0.4788 AUPRC=0.4288 ROC_AUC=0.8031 thr=0.036 lr=1.00e-03 | best_AUPRC=0.4288 (ep7)
ep008 | tr_loss=0.5180 val_loss=0.5308 F1=0.4826 AUPRC=0.4298 ROC_AUC=0.8007 thr=0.209 lr=1.00e-03 | best_AUPRC=0.4298 (ep8)
ep009 | tr_loss=0.5129 val_loss=0.5397 F1=0.4829 AUPRC=0.4289 ROC_AUC=0.7985 thr=0.029 lr=1.00e-03 | best_AUPRC=0.4298 (ep8)
ep010 | tr_loss=0.5101 val_loss=0.5297 F1=0.4868 AUPRC=0.4335 ROC_AUC=0.8025 thr=0.094 lr=1.00e-03 | best_AUPRC=0.4335 (ep10)
ep011 | tr_loss=0.5081 val_loss=0.5304 F1=0.4903 AUPRC=0.4334 ROC_AUC=0.7992 thr=0.051 lr=1.00e-03 | best_AUPRC=0.4335 (ep10)
ep012 | tr_loss=0.5058 val_loss=0.5216 F1=0.4897 AUPRC=0.4351 ROC_AUC=0.8035 thr=0.113 lr=1.00e-03 | best_AUPRC=0.4351 (ep12)
ep013 | tr_loss=0.5027 val_loss=0.5176 F1=0.4932 AUPRC=0.4346 ROC_AUC=0.8024 thr=0.251 lr=1.00e-03 | best_AUPRC=0.4351 (ep12)
ep014 | tr_loss=0.5028 val_loss=0.5147 F1=0.4967 AUPRC=0.4365 ROC_AUC=0.8014 thr=0.156 lr=1.00e-03 | best_AUPRC=0.4365 (ep14)
ep015 | tr_loss=0.4976 val_loss=0.5158 F1=0.4905 AUPRC=0.4339 ROC_AUC=0.8034 thr=0.116 lr=1.00e-03 | best_AUPRC=0.4365 (ep14)
ep016 | tr_loss=0.4989 val_loss=0.5122 F1=0.4951 AUPRC=0.4360 ROC_AUC=0.8029 thr=0.394 lr=1.00e-03 | best_AUPRC=0.4365 (ep14)
ep017 | tr_loss=0.4942 val_loss=0.5188 F1=0.4955 AUPRC=0.4354 ROC_AUC=0.7991 thr=0.086 lr=1.00e-03 | best_AUPRC=0.4365 (ep14)
ep018 | tr_loss=0.4948 val_loss=0.5359 F1=0.4970 AUPRC=0.4407 ROC_AUC=0.8051 thr=0.010 lr=1.00e-03 | best_AUPRC=0.4407 (ep18)
ep019 | tr_loss=0.4919 val_loss=0.5209 F1=0.4973 AUPRC=0.4409 ROC_AUC=0.8076 thr=0.058 lr=1.00e-03 | best_AUPRC=0.4409 (ep19)
ep020 | tr_loss=0.4911 val_loss=0.5222 F1=0.4981 AUPRC=0.4400 ROC_AUC=0.8062 thr=0.037 lr=1.00e-03 | best_AUPRC=0.4409 (ep19)
ep021 | tr_loss=0.4887 val_loss=0.5148 F1=0.4977 AUPRC=0.4412 ROC_AUC=0.8104 thr=0.136 lr=1.00e-03 | best_AUPRC=0.4412 (ep21)
ep022 | tr_loss=0.4889 val_loss=0.5075 F1=0.4973 AUPRC=0.4389 ROC_AUC=0.8094 thr=0.265 lr=1.00e-03 | best_AUPRC=0.4412 (ep21)
ep023 | tr_loss=0.4871 val_loss=0.5186 F1=0.4997 AUPRC=0.4435 ROC_AUC=0.8142 thr=0.037 lr=1.00e-03 | best_AUPRC=0.4435 (ep23)
ep024 | tr_loss=0.4829 val_loss=0.5073 F1=0.4990 AUPRC=0.4394 ROC_AUC=0.8099 thr=0.220 lr=1.00e-03 | best_AUPRC=0.4435 (ep23)
ep025 | tr_loss=0.4839 val_loss=0.5307 F1=0.4999 AUPRC=0.4425 ROC_AUC=0.8096 thr=0.003 lr=1.00e-03 | best_AUPRC=0.4435 (ep23)
ep026 | tr_loss=0.4831 val_loss=0.5208 F1=0.4999 AUPRC=0.4417 ROC_AUC=0.8084 thr=
```

r=0.038 lr=1.00e-03 | best_AUPRC=0.4435 (ep23)
 ep027 | tr_loss=0.4839 val_loss=0.5104 F1=0.5019 AUPRC=0.4399 ROC_AUC=0.8093 th
 r=0.086 lr=1.00e-03 | best_AUPRC=0.4435 (ep23)
 ep028 | tr_loss=0.4802 val_loss=0.5086 F1=0.4973 AUPRC=0.4384 ROC_AUC=0.8100 th
 r=0.358 lr=1.00e-03 | best_AUPRC=0.4435 (ep23)
 ep029 | tr_loss=0.4774 val_loss=0.5197 F1=0.5013 AUPRC=0.4443 ROC_AUC=0.8105 th
 r=0.028 lr=1.00e-03 | best_AUPRC=0.4443 (ep29)
 ep030 | tr_loss=0.4788 val_loss=0.5079 F1=0.5001 AUPRC=0.4400 ROC_AUC=0.8094 th
 r=0.224 lr=5.00e-04 | best_AUPRC=0.4443 (ep29)
 ep031 | tr_loss=0.4723 val_loss=0.5080 F1=0.5020 AUPRC=0.4418 ROC_AUC=0.8119 th
 r=0.201 lr=5.00e-04 | best_AUPRC=0.4443 (ep29)
 ep032 | tr_loss=0.4724 val_loss=0.5063 F1=0.5010 AUPRC=0.4416 ROC_AUC=0.8107 th
 r=0.126 lr=5.00e-04 | best_AUPRC=0.4443 (ep29)
 ep033 | tr_loss=0.4730 val_loss=0.5190 F1=0.5010 AUPRC=0.4432 ROC_AUC=0.8117 th
 r=0.035 lr=5.00e-04 | best_AUPRC=0.4443 (ep29)
 ep034 | tr_loss=0.4703 val_loss=0.5104 F1=0.5011 AUPRC=0.4434 ROC_AUC=0.8120 th
 r=0.124 lr=5.00e-04 | best_AUPRC=0.4443 (ep29)
 ep035 | tr_loss=0.4706 val_loss=0.5085 F1=0.5031 AUPRC=0.4433 ROC_AUC=0.8127 th
 r=0.083 lr=5.00e-04 | best_AUPRC=0.4443 (ep29)
 ep036 | tr_loss=0.4702 val_loss=0.5111 F1=0.5008 AUPRC=0.4421 ROC_AUC=0.8118 th
 r=0.106 lr=5.00e-04 | best_AUPRC=0.4443 (ep29)
 ep037 | tr_loss=0.4703 val_loss=0.5221 F1=0.5015 AUPRC=0.4453 ROC_AUC=0.8116 th
 r=0.016 lr=5.00e-04 | best_AUPRC=0.4453 (ep37)
 ep038 | tr_loss=0.4686 val_loss=0.5274 F1=0.5011 AUPRC=0.4449 ROC_AUC=0.8115 th
 r=0.018 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep039 | tr_loss=0.4657 val_loss=0.5069 F1=0.5016 AUPRC=0.4430 ROC_AUC=0.8114 th
 r=0.074 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep040 | tr_loss=0.4640 val_loss=0.5191 F1=0.5029 AUPRC=0.4447 ROC_AUC=0.8112 th
 r=0.048 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep041 | tr_loss=0.4663 val_loss=0.5130 F1=0.5028 AUPRC=0.4445 ROC_AUC=0.8119 th
 r=0.051 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep042 | tr_loss=0.4654 val_loss=0.5127 F1=0.5034 AUPRC=0.4447 ROC_AUC=0.8124 th
 r=0.055 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep043 | tr_loss=0.4635 val_loss=0.5155 F1=0.5031 AUPRC=0.4451 ROC_AUC=0.8126 th
 r=0.046 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep044 | tr_loss=0.4652 val_loss=0.5050 F1=0.5030 AUPRC=0.4432 ROC_AUC=0.8130 th
 r=0.171 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep045 | tr_loss=0.4637 val_loss=0.5087 F1=0.5020 AUPRC=0.4442 ROC_AUC=0.8126 th
 r=0.086 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep046 | tr_loss=0.4634 val_loss=0.5097 F1=0.5026 AUPRC=0.4446 ROC_AUC=0.8133 th
 r=0.129 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ep047 | tr_loss=0.4632 val_loss=0.5094 F1=0.5030 AUPRC=0.4443 ROC_AUC=0.8128 th
 r=0.085 lr=2.50e-04 | best_AUPRC=0.4453 (ep37)
 ■ early stop @ ep47 (no improv 10 epochs)

simple | DiceLoss

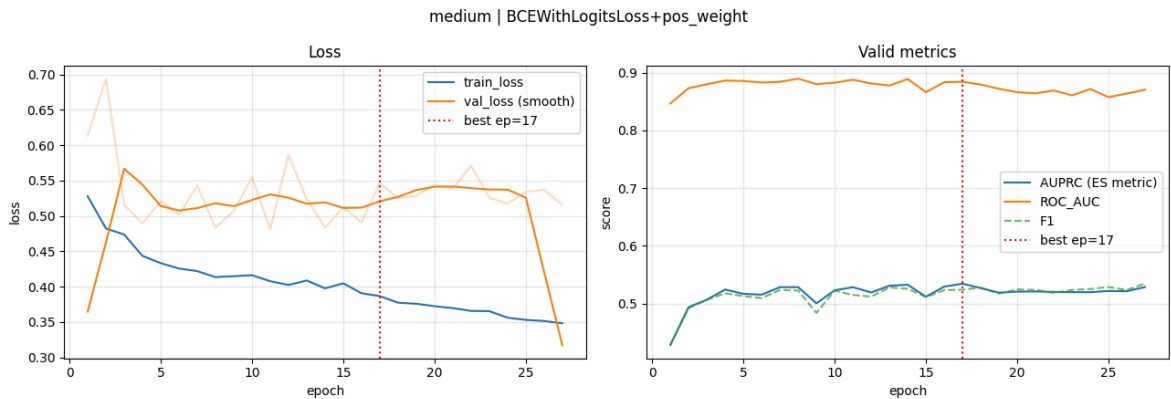


```
weights -> best_mlp_simple_DiceLoss.pth
history -> history_simple_DiceLoss.json
VAL : {'F1': 0.5015299026425591, 'Precision': 0.6991081814656843, 'Recall': 0.391021470396877, 'ROC_AUC': 0.8115569410577571, 'AUPRC': 0.4452762643542555}
TEST : {'F1': 0.453680981595092, 'Precision': 0.6021986970684039, 'Recall': 0.36392716535433073, 'ROC_AUC': 0.7938743558398651, 'AUPRC': 0.3622779875567114} (thr=0.0160)
```

===== MEDIUM | BCEWithLogitsLoss+pos_weight | dropout=0.15 =====

```
ep001 | tr_loss=0.5279 val_loss=0.6134 F1=0.4280 AUPRC=0.4283 ROC_AUC=0.8466 thr=0.862 lr=1.00e-03 | best_AUPRC=0.4283 (ep1)
ep002 | tr_loss=0.4823 val_loss=0.6942 F1=0.4919 AUPRC=0.4936 ROC_AUC=0.8731 thr=0.850 lr=1.00e-03 | best_AUPRC=0.4936 (ep2)
ep003 | tr_loss=0.4737 val_loss=0.5155 F1=0.5063 AUPRC=0.5066 ROC_AUC=0.8799 thr=0.591 lr=1.00e-03 | best_AUPRC=0.5066 (ep3)
ep004 | tr_loss=0.4435 val_loss=0.4898 F1=0.5176 AUPRC=0.5242 ROC_AUC=0.8863 thr=0.764 lr=1.00e-03 | best_AUPRC=0.5242 (ep4)
ep005 | tr_loss=0.4333 val_loss=0.5207 F1=0.5127 AUPRC=0.5167 ROC_AUC=0.8855 thr=0.787 lr=1.00e-03 | best_AUPRC=0.5242 (ep4)
ep006 | tr_loss=0.4256 val_loss=0.5015 F1=0.5092 AUPRC=0.5153 ROC_AUC=0.8829 thr=0.656 lr=1.00e-03 | best_AUPRC=0.5242 (ep4)
ep007 | tr_loss=0.4220 val_loss=0.5429 F1=0.5234 AUPRC=0.5283 ROC_AUC=0.8839 thr=0.429 lr=1.00e-03 | best_AUPRC=0.5283 (ep7)
ep008 | tr_loss=0.4135 val_loss=0.4836 F1=0.5226 AUPRC=0.5284 ROC_AUC=0.8896 thr=0.780 lr=1.00e-03 | best_AUPRC=0.5284 (ep8)
ep009 | tr_loss=0.4148 val_loss=0.5069 F1=0.4837 AUPRC=0.5004 ROC_AUC=0.8801 thr=0.869 lr=1.00e-03 | best_AUPRC=0.5284 (ep8)
ep010 | tr_loss=0.4162 val_loss=0.5550 F1=0.5223 AUPRC=0.5229 ROC_AUC=0.8825 thr=0.429 lr=1.00e-03 | best_AUPRC=0.5284 (ep8)
ep011 | tr_loss=0.4076 val_loss=0.4810 F1=0.5148 AUPRC=0.5282 ROC_AUC=0.8877 thr=0.843 lr=1.00e-03 | best_AUPRC=0.5284 (ep8)
ep012 | tr_loss=0.4023 val_loss=0.5861 F1=0.5120 AUPRC=0.5191 ROC_AUC=0.8811 thr=0.441 lr=1.00e-03 | best_AUPRC=0.5284 (ep8)
ep013 | tr_loss=0.4087 val_loss=0.5235 F1=0.5277 AUPRC=0.5308 ROC_AUC=0.8778 thr=0.586 lr=1.00e-03 | best_AUPRC=0.5308 (ep13)
ep014 | tr_loss=0.3976 val_loss=0.4834 F1=0.5256 AUPRC=0.5325 ROC_AUC=0.8889 thr=0.786 lr=1.00e-03 | best_AUPRC=0.5325 (ep14)
ep015 | tr_loss=0.4046 val_loss=0.5123 F1=0.5112 AUPRC=0.5117 ROC_AUC=0.8662 thr=0.754 lr=1.00e-03 | best_AUPRC=0.5325 (ep14)
ep016 | tr_loss=0.3906 val_loss=0.4911 F1=0.5231 AUPRC=0.5292 ROC_AUC=0.8835 thr=0.625 lr=1.00e-03 | best_AUPRC=0.5325 (ep14)
ep017 | tr_loss=0.3865 val_loss=0.5467 F1=0.5238 AUPRC=0.5343 ROC_AUC=0.8841 thr=0.529 lr=5.00e-04 | best_AUPRC=0.5343 (ep17)
ep018 | tr_loss=0.3773 val_loss=0.5251 F1=0.5276 AUPRC=0.5266 ROC_AUC=0.8794 thr=0.664 lr=5.00e-04 | best_AUPRC=0.5343 (ep17)
ep019 | tr_loss=0.3758 val_loss=0.5285 F1=0.5171 AUPRC=0.5188 ROC_AUC=0.8720 thr=0.576 lr=5.00e-04 | best_AUPRC=0.5343 (ep17)
ep020 | tr_loss=0.3723 val_loss=0.5437 F1=0.5245 AUPRC=0.5206 ROC_AUC=0.8662 thr=0.741 lr=5.00e-04 | best_AUPRC=0.5343 (ep17)
ep021 | tr_loss=0.3696 val_loss=0.5389 F1=0.5235 AUPRC=0.5208 ROC_AUC=0.8641 thr=0.607 lr=5.00e-04 | best_AUPRC=0.5343 (ep17)
ep022 | tr_loss=0.3656 val_loss=0.5711 F1=0.5182 AUPRC=0.5200 ROC_AUC=0.8691 thr=0.530 lr=5.00e-04 | best_AUPRC=0.5343 (ep17)
ep023 | tr_loss=0.3653 val_loss=0.5259 F1=0.5234 AUPRC=0.5197 ROC_AUC=0.8606 thr=0.674 lr=2.50e-04 | best_AUPRC=0.5343 (ep17)
ep024 | tr_loss=0.3560 val_loss=0.5173 F1=0.5250 AUPRC=0.5196 ROC_AUC=0.8717 thr=0.697 lr=2.50e-04 | best_AUPRC=0.5343 (ep17)
ep025 | tr_loss=0.3530 val_loss=0.5341 F1=0.5285 AUPRC=0.5217 ROC_AUC=0.8576 thr=0.698 lr=2.50e-04 | best_AUPRC=0.5343 (ep17)
ep026 | tr_loss=0.3514 val_loss=0.5372 F1=0.5234 AUPRC=0.5212 ROC_AUC=0.8637 thr=
```

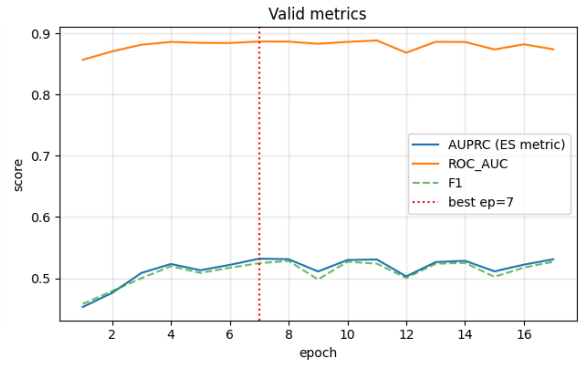
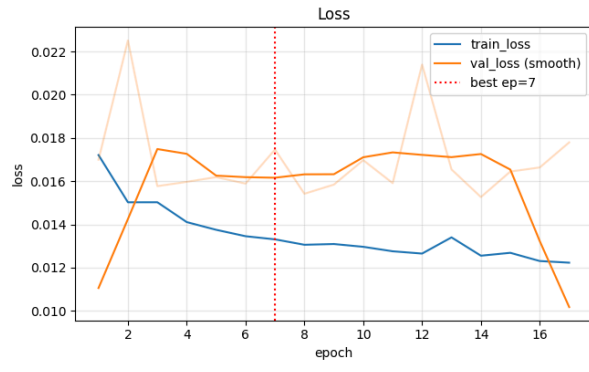
r=0.725 lr=2.50e-04 | best_AUPRC=0.5343 (ep17)
 ep027 | tr_loss=0.3481 val_loss=0.5154 F1=0.5345 AUPRC=0.5285 ROC_AUC=0.8705 th
 r=0.748 lr=2.50e-04 | best_AUPRC=0.5343 (ep17)
 ■ early stop @ ep27 (no improv 10 epochs)



weights -> best_mlp_medium_BCEWithLogitsLoss_pos_weight.pth
 history -> history_medium_BCEWithLogitsLoss_pos_weight.json
 VAL : {'F1': 0.5238394766637313, 'Precision': 0.6237267825044938, 'Recall': 0.4515289525048796, 'ROC_AUC': 0.8841233319251904, 'AUPRC': 0.5343455440878301}
 TEST : {'F1': 0.4483338000560067, 'Precision': 0.5201429499675114, 'Recall': 0.3939468503937008, 'ROC_AUC': 0.8407000273775393, 'AUPRC': 0.41565776284013617} (t
 hr=0.5292)

===== MEDIUM | FocalLoss(alpha=0.75,gamma=2.0) | dropout=0.15 =====
 ep001 | tr_loss=0.0172 val_loss=0.0170 F1=0.4580 AUPRC=0.4530 ROC_AUC=0.8569 th
 r=0.452 lr=1.00e-03 | best_AUPRC=0.4530 (ep1)
 ep002 | tr_loss=0.0150 val_loss=0.0225 F1=0.4790 AUPRC=0.4761 ROC_AUC=0.8706 th
 r=0.691 lr=1.00e-03 | best_AUPRC=0.4761 (ep2)
 ep003 | tr_loss=0.0150 val_loss=0.0158 F1=0.5001 AUPRC=0.5088 ROC_AUC=0.8815 th
 r=0.527 lr=1.00e-03 | best_AUPRC=0.5088 (ep3)
 ep004 | tr_loss=0.0141 val_loss=0.0160 F1=0.5197 AUPRC=0.5233 ROC_AUC=0.8861 th
 r=0.442 lr=1.00e-03 | best_AUPRC=0.5233 (ep4)
 ep005 | tr_loss=0.0137 val_loss=0.0162 F1=0.5088 AUPRC=0.5131 ROC_AUC=0.8846 th
 r=0.475 lr=1.00e-03 | best_AUPRC=0.5233 (ep4)
 ep006 | tr_loss=0.0135 val_loss=0.0159 F1=0.5171 AUPRC=0.5219 ROC_AUC=0.8843 th
 r=0.431 lr=1.00e-03 | best_AUPRC=0.5233 (ep4)
 ep007 | tr_loss=0.0133 val_loss=0.0175 F1=0.5247 AUPRC=0.5320 ROC_AUC=0.8868 th
 r=0.373 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep008 | tr_loss=0.0131 val_loss=0.0154 F1=0.5283 AUPRC=0.5311 ROC_AUC=0.8866 th
 r=0.524 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep009 | tr_loss=0.0131 val_loss=0.0158 F1=0.4980 AUPRC=0.5111 ROC_AUC=0.8831 th
 r=0.451 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep010 | tr_loss=0.0130 val_loss=0.0170 F1=0.5271 AUPRC=0.5298 ROC_AUC=0.8862 th
 r=0.379 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep011 | tr_loss=0.0128 val_loss=0.0159 F1=0.5238 AUPRC=0.5307 ROC_AUC=0.8885 th
 r=0.474 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep012 | tr_loss=0.0126 val_loss=0.0214 F1=0.5007 AUPRC=0.5031 ROC_AUC=0.8685 th
 r=0.304 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep013 | tr_loss=0.0134 val_loss=0.0165 F1=0.5237 AUPRC=0.5265 ROC_AUC=0.8863 th
 r=0.411 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep014 | tr_loss=0.0126 val_loss=0.0153 F1=0.5251 AUPRC=0.5285 ROC_AUC=0.8860 th
 r=0.493 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep015 | tr_loss=0.0127 val_loss=0.0164 F1=0.5024 AUPRC=0.5112 ROC_AUC=0.8737 th
 r=0.462 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep016 | tr_loss=0.0123 val_loss=0.0166 F1=0.5175 AUPRC=0.5223 ROC_AUC=0.8821 th
 r=0.404 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ep017 | tr_loss=0.0122 val_loss=0.0178 F1=0.5270 AUPRC=0.5310 ROC_AUC=0.8740 th
 r=0.382 lr=1.00e-03 | best_AUPRC=0.5320 (ep7)
 ■ early stop @ ep17 (no improv 10 epochs)

medium | FocalLoss(alpha=0.75,gamma=2.0)



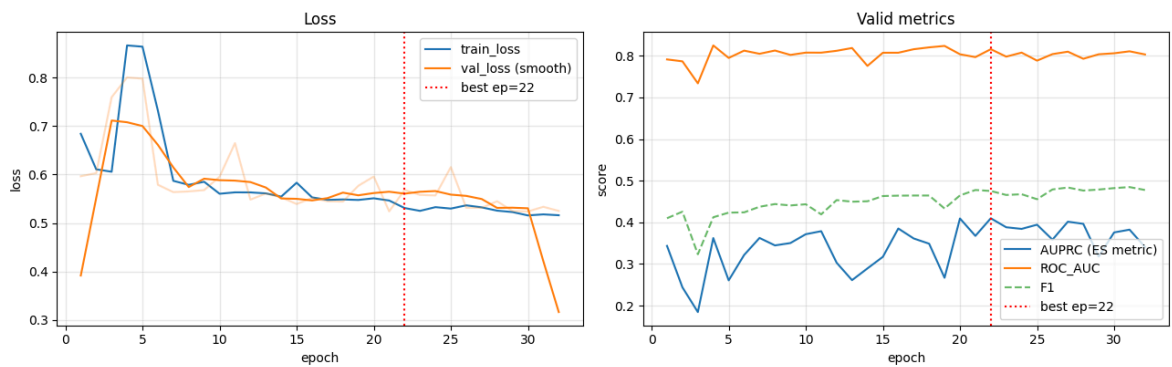
```
weights -> best_mlp_medium_FocalLoss_alpha075_gamma20.pth
history -> history_medium_FocalLoss_alpha075_gamma20.json
VAL : {'F1': 0.5247158002299144, 'Precision': 0.6382846488502175, 'Recall': 0.
4454565170245066, 'ROC_AUC': 0.8867588806227507, 'AUPRC': 0.5319754277489046}
TEST : {'F1': 0.4430203399275564, 'Precision': 0.5105973025048169, 'Recall': 0.
391240157480315, 'ROC_AUC': 0.8558517427622216, 'AUPRC': 0.41956995144724807} (th
r=0.3729)
```

===== MEDIUM | DiceLoss | dropout=0.15 =====

```
ep001 | tr_loss=0.6839 val_loss=0.5965 F1=0.4099 AUPRC=0.3433 ROC_AUC=0.7914 th
r=0.262 lr=1.00e-03 | best_AUPRC=0.3433 (ep1)
ep002 | tr_loss=0.6107 val_loss=0.6027 F1=0.4256 AUPRC=0.2437 ROC_AUC=0.7868 th
r=0.999 lr=1.00e-03 | best_AUPRC=0.3433 (ep1)
ep003 | tr_loss=0.6057 val_loss=0.7593 F1=0.3233 AUPRC=0.1844 ROC_AUC=0.7340 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3433 (ep1)
ep004 | tr_loss=0.8661 val_loss=0.8001 F1=0.4122 AUPRC=0.3622 ROC_AUC=0.8250 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3622 (ep4)
ep005 | tr_loss=0.8636 val_loss=0.7976 F1=0.4233 AUPRC=0.2606 ROC_AUC=0.7949 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3622 (ep4)
ep006 | tr_loss=0.7317 val_loss=0.5788 F1=0.4239 AUPRC=0.3215 ROC_AUC=0.8125 th
r=0.933 lr=1.00e-03 | best_AUPRC=0.3622 (ep4)
ep007 | tr_loss=0.5869 val_loss=0.5637 F1=0.4374 AUPRC=0.3625 ROC_AUC=0.8049 th
r=0.373 lr=1.00e-03 | best_AUPRC=0.3625 (ep7)
ep008 | tr_loss=0.5787 val_loss=0.5651 F1=0.4439 AUPRC=0.3445 ROC_AUC=0.8128 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3625 (ep7)
ep009 | tr_loss=0.5850 val_loss=0.5678 F1=0.4405 AUPRC=0.3502 ROC_AUC=0.8022 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3625 (ep7)
ep010 | tr_loss=0.5603 val_loss=0.5954 F1=0.4434 AUPRC=0.3714 ROC_AUC=0.8077 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3714 (ep10)
ep011 | tr_loss=0.5632 val_loss=0.6647 F1=0.4192 AUPRC=0.3787 ROC_AUC=0.8076 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3787 (ep11)
ep012 | tr_loss=0.5631 val_loss=0.5482 F1=0.4534 AUPRC=0.3030 ROC_AUC=0.8121 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3787 (ep11)
ep013 | tr_loss=0.5611 val_loss=0.5607 F1=0.4496 AUPRC=0.2611 ROC_AUC=0.8188 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3787 (ep11)
ep014 | tr_loss=0.5544 val_loss=0.5530 F1=0.4506 AUPRC=0.2893 ROC_AUC=0.7759 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3787 (ep11)
ep015 | tr_loss=0.5831 val_loss=0.5392 F1=0.4631 AUPRC=0.3171 ROC_AUC=0.8074 th
r=0.005 lr=1.00e-03 | best_AUPRC=0.3787 (ep11)
ep016 | tr_loss=0.5528 val_loss=0.5516 F1=0.4641 AUPRC=0.3851 ROC_AUC=0.8075 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3851 (ep16)
ep017 | tr_loss=0.5476 val_loss=0.5448 F1=0.4644 AUPRC=0.3612 ROC_AUC=0.8159 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3851 (ep16)
ep018 | tr_loss=0.5486 val_loss=0.5442 F1=0.4644 AUPRC=0.3487 ROC_AUC=0.8204 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3851 (ep16)
ep019 | tr_loss=0.5475 val_loss=0.5767 F1=0.4333 AUPRC=0.2668 ROC_AUC=0.8238 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3851 (ep16)
ep020 | tr_loss=0.5509 val_loss=0.5958 F1=0.4646 AUPRC=0.4093 ROC_AUC=0.8038 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.4093 (ep20)
ep021 | tr_loss=0.5464 val_loss=0.5240 F1=0.4777 AUPRC=0.3674 ROC_AUC=0.7969 th
r=0.001 lr=1.00e-03 | best_AUPRC=0.4093 (ep20)
ep022 | tr_loss=0.5309 val_loss=0.5680 F1=0.4754 AUPRC=0.4097 ROC_AUC=0.8160 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.4097 (ep22)
ep023 | tr_loss=0.5250 val_loss=0.5579 F1=0.4657 AUPRC=0.3881 ROC_AUC=0.7981 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.4097 (ep22)
ep024 | tr_loss=0.5327 val_loss=0.5568 F1=0.4675 AUPRC=0.3842 ROC_AUC=0.8078 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.4097 (ep22)
ep025 | tr_loss=0.5297 val_loss=0.6152 F1=0.4555 AUPRC=0.3943 ROC_AUC=0.7885 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.4097 (ep22)
ep026 | tr_loss=0.5364 val_loss=0.5314 F1=0.4790 AUPRC=0.3584 ROC_AUC=0.8041 th
```


r=0.000 lr=1.00e-03 | best_AUPRC=0.4097 (ep22)
 ep027 | tr_loss=0.5323 val_loss=0.5312 F1=0.4836 AUPRC=0.4015 ROC_AUC=0.8100 th
 r=0.000 lr=5.00e-04 | best_AUPRC=0.4097 (ep22)
 ep028 | tr_loss=0.5255 val_loss=0.5448 F1=0.4763 AUPRC=0.3963 ROC_AUC=0.7930 th
 r=0.000 lr=5.00e-04 | best_AUPRC=0.4097 (ep22)
 ep029 | tr_loss=0.5226 val_loss=0.5247 F1=0.4788 AUPRC=0.3180 ROC_AUC=0.8038 th
 r=1.000 lr=5.00e-04 | best_AUPRC=0.4097 (ep22)
 ep030 | tr_loss=0.5156 val_loss=0.5238 F1=0.4822 AUPRC=0.3757 ROC_AUC=0.8062 th
 r=0.000 lr=5.00e-04 | best_AUPRC=0.4097 (ep22)
 ep031 | tr_loss=0.5178 val_loss=0.5334 F1=0.4847 AUPRC=0.3824 ROC_AUC=0.8109 th
 r=0.000 lr=5.00e-04 | best_AUPRC=0.4097 (ep22)
 ep032 | tr_loss=0.5160 val_loss=0.5249 F1=0.4777 AUPRC=0.3408 ROC_AUC=0.8034 th
 r=1.000 lr=5.00e-04 | best_AUPRC=0.4097 (ep22)
 ■ early stop @ ep32 (no improv 10 epochs)

medium | DiceLoss



```

weights -> best_mlp_medium_DiceLoss.pth
history -> history_medium_DiceLoss.json
VAL : {'F1': 0.4753531277025079, 'Precision': 0.7086377309840997, 'Recall': 0.
3576230752548254, 'ROC_AUC': 0.8159834498409027, 'AUPRC': 0.40972097289944626}
TEST : {'F1': 0.43217665615141954, 'Precision': 0.6019332161687171, 'Recall':
0.33710629921259844, 'ROC_AUC': 0.8053330712148123, 'AUPRC': 0.3563161763797823}
(thr=0.0000)

```

===== ADVANCED | BCEWithLogitsLoss+pos_weight | dropout=0.15 =====

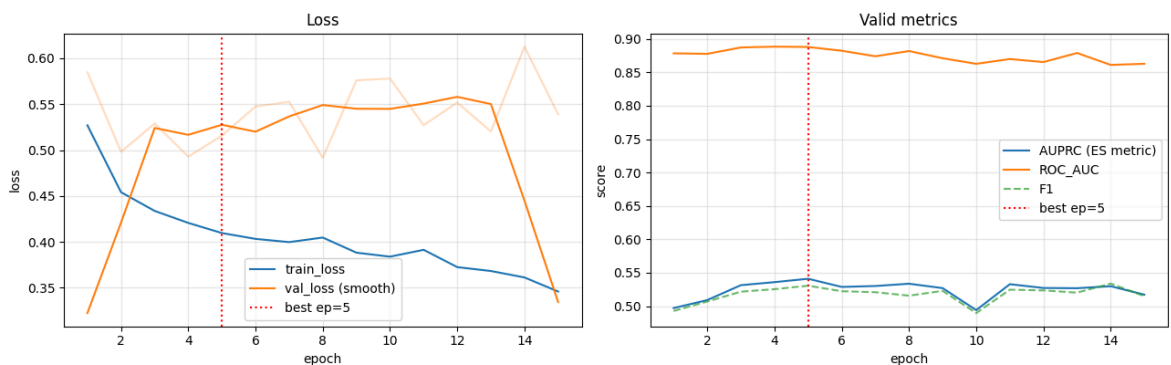
```

ep001 | tr_loss=0.5269 val_loss=0.5848 F1=0.4929 AUPRC=0.4972 ROC_AUC=0.8785 th
r=0.415 lr=1.00e-03 | best_AUPRC=0.4972 (ep1)
ep002 | tr_loss=0.4540 val_loss=0.4981 F1=0.5070 AUPRC=0.5091 ROC_AUC=0.8777 th
r=0.746 lr=1.00e-03 | best_AUPRC=0.5091 (ep2)
ep003 | tr_loss=0.4337 val_loss=0.5290 F1=0.5217 AUPRC=0.5315 ROC_AUC=0.8873 th
r=0.484 lr=1.00e-03 | best_AUPRC=0.5315 (ep3)
ep004 | tr_loss=0.4206 val_loss=0.4929 F1=0.5255 AUPRC=0.5360 ROC_AUC=0.8885 th
r=0.652 lr=1.00e-03 | best_AUPRC=0.5360 (ep4)
ep005 | tr_loss=0.4096 val_loss=0.5157 F1=0.5306 AUPRC=0.5411 ROC_AUC=0.8880 th
r=0.552 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep006 | tr_loss=0.4032 val_loss=0.5475 F1=0.5224 AUPRC=0.5288 ROC_AUC=0.8824 th
r=0.555 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep007 | tr_loss=0.3997 val_loss=0.5526 F1=0.5209 AUPRC=0.5302 ROC_AUC=0.8741 th
r=0.787 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep008 | tr_loss=0.4047 val_loss=0.4915 F1=0.5157 AUPRC=0.5336 ROC_AUC=0.8819 th
r=0.743 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep009 | tr_loss=0.3882 val_loss=0.5760 F1=0.5229 AUPRC=0.5271 ROC_AUC=0.8712 th
r=0.519 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep010 | tr_loss=0.3838 val_loss=0.5780 F1=0.4897 AUPRC=0.4941 ROC_AUC=0.8628 th
r=0.868 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep011 | tr_loss=0.3913 val_loss=0.5272 F1=0.5247 AUPRC=0.5329 ROC_AUC=0.8699 th
r=0.690 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep012 | tr_loss=0.3724 val_loss=0.5518 F1=0.5236 AUPRC=0.5272 ROC_AUC=0.8654 th
r=0.550 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep013 | tr_loss=0.3682 val_loss=0.5203 F1=0.5203 AUPRC=0.5269 ROC_AUC=0.8790 th
r=0.587 lr=1.00e-03 | best_AUPRC=0.5411 (ep5)
ep014 | tr_loss=0.3612 val_loss=0.6127 F1=0.5338 AUPRC=0.5299 ROC_AUC=0.8612 th
r=0.528 lr=5.00e-04 | best_AUPRC=0.5411 (ep5)
ep015 | tr_loss=0.3458 val_loss=0.5389 F1=0.5151 AUPRC=0.5172 ROC_AUC=0.8629 th
r=0.744 lr=5.00e-04 | best_AUPRC=0.5411 (ep5)

```

■ early stop @ ep15 (no improv 10 epochs)

advanced | BCEWithLogitsLoss+pos_weight



```

weights -> best_mlp_advanced_BCEWithLogitsLoss_pos_weight.pth
history -> history_advanced_BCEWithLogitsLoss_pos_weight.json
VAL : {'F1': 0.5306331709817995, 'Precision': 0.6486994672516453, 'Recall': 0.
44892648015614833, 'ROC_AUC': 0.8880116029080106, 'AUPRC': 0.5410537763717399}
TEST : {'F1': 0.4421315570358035, 'Precision': 0.5070019096117123, 'Recall': 0.
39197834645669294, 'ROC_AUC': 0.8565033380368181, 'AUPRC': 0.4157750383613668} (t
hr=0.5518)

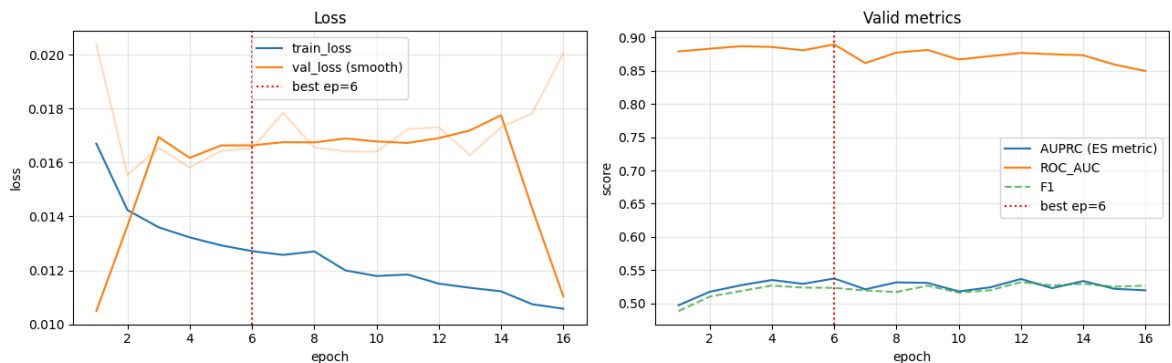
```

```

===== ADVANCED | FocalLoss(alpha=0.75,gamma=2.0) | dropout=0.15 =====
ep001 | tr_loss=0.0167 val_loss=0.0204 F1=0.4887 AUPRC=0.4973 ROC_AUC=0.8788 th
r=0.314 lr=1.00e-03 | best_AUPRC=0.4973 (ep1)
ep002 | tr_loss=0.0142 val_loss=0.0155 F1=0.5104 AUPRC=0.5175 ROC_AUC=0.8829 th
r=0.498 lr=1.00e-03 | best_AUPRC=0.5175 (ep2)
ep003 | tr_loss=0.0136 val_loss=0.0165 F1=0.5187 AUPRC=0.5274 ROC_AUC=0.8867 th
r=0.401 lr=1.00e-03 | best_AUPRC=0.5274 (ep3)
ep004 | tr_loss=0.0132 val_loss=0.0158 F1=0.5270 AUPRC=0.5353 ROC_AUC=0.8855 th
r=0.424 lr=1.00e-03 | best_AUPRC=0.5353 (ep4)
ep005 | tr_loss=0.0129 val_loss=0.0164 F1=0.5239 AUPRC=0.5295 ROC_AUC=0.8805 th
r=0.407 lr=1.00e-03 | best_AUPRC=0.5353 (ep4)
ep006 | tr_loss=0.0127 val_loss=0.0165 F1=0.5235 AUPRC=0.5376 ROC_AUC=0.8893 th
r=0.430 lr=1.00e-03 | best_AUPRC=0.5376 (ep6)
ep007 | tr_loss=0.0126 val_loss=0.0178 F1=0.5196 AUPRC=0.5213 ROC_AUC=0.8613 th
r=0.490 lr=1.00e-03 | best_AUPRC=0.5376 (ep6)
ep008 | tr_loss=0.0127 val_loss=0.0166 F1=0.5173 AUPRC=0.5317 ROC_AUC=0.8771 th
r=0.419 lr=5.00e-04 | best_AUPRC=0.5376 (ep6)
ep009 | tr_loss=0.0120 val_loss=0.0164 F1=0.5267 AUPRC=0.5310 ROC_AUC=0.8810 th
r=0.445 lr=5.00e-04 | best_AUPRC=0.5376 (ep6)
ep010 | tr_loss=0.0118 val_loss=0.0164 F1=0.5164 AUPRC=0.5182 ROC_AUC=0.8667 th
r=0.491 lr=5.00e-04 | best_AUPRC=0.5376 (ep6)
ep011 | tr_loss=0.0118 val_loss=0.0172 F1=0.5198 AUPRC=0.5242 ROC_AUC=0.8716 th
r=0.410 lr=5.00e-04 | best_AUPRC=0.5376 (ep6)
ep012 | tr_loss=0.0115 val_loss=0.0173 F1=0.5320 AUPRC=0.5368 ROC_AUC=0.8765 th
r=0.419 lr=5.00e-04 | best_AUPRC=0.5376 (ep6)
ep013 | tr_loss=0.0114 val_loss=0.0163 F1=0.5272 AUPRC=0.5230 ROC_AUC=0.8746 th
r=0.460 lr=5.00e-04 | best_AUPRC=0.5376 (ep6)
ep014 | tr_loss=0.0112 val_loss=0.0173 F1=0.5295 AUPRC=0.5338 ROC_AUC=0.8731 th
r=0.399 lr=2.50e-04 | best_AUPRC=0.5376 (ep6)
ep015 | tr_loss=0.0107 val_loss=0.0178 F1=0.5254 AUPRC=0.5221 ROC_AUC=0.8591 th
r=0.475 lr=2.50e-04 | best_AUPRC=0.5376 (ep6)
ep016 | tr_loss=0.0106 val_loss=0.0200 F1=0.5269 AUPRC=0.5198 ROC_AUC=0.8494 th
r=0.428 lr=2.50e-04 | best_AUPRC=0.5376 (ep6)
■ early stop @ ep16 (no improv 10 epochs)

```

advanced | FocalLoss(alpha=0.75,gamma=2.0)



```

weights -> best_mlp_advanced_FocalLoss_alpha075_gamma20.pth
history -> history_advanced_FocalLoss_alpha075_gamma20.json
VAL : {'F1': 0.5234985273402484, 'Precision': 0.6391494684177611, 'Recall': 0.
4432877900672305, 'ROC_AUC': 0.8893390880600692, 'AUPRC': 0.5375906960834388}
TEST : {'F1': 0.44485555713675823, 'Precision': 0.5275059061761728, 'Recall':
0.38459645669291337, 'ROC_AUC': 0.8579985185214465, 'AUPRC': 0.39665307403651284}
(thr=0.4300)

```

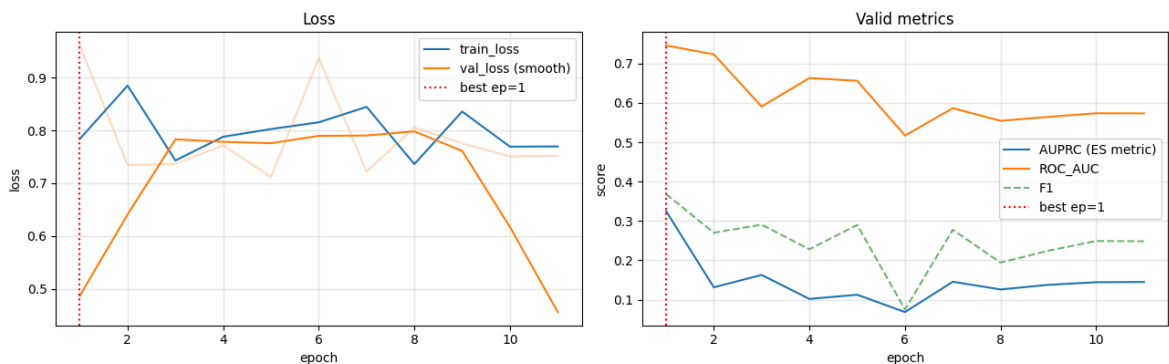
===== ADVANCED | DiceLoss | dropout=0.15 =====

```

ep001 | tr_loss=0.7841 val_loss=0.9615 F1=0.3681 AUPRC=0.3253 ROC_AUC=0.7462 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep002 | tr_loss=0.8853 val_loss=0.7345 F1=0.2701 AUPRC=0.1314 ROC_AUC=0.7235 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep003 | tr_loss=0.7433 val_loss=0.7368 F1=0.2907 AUPRC=0.1629 ROC_AUC=0.5909 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep004 | tr_loss=0.7882 val_loss=0.7721 F1=0.2281 AUPRC=0.1019 ROC_AUC=0.6633 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep005 | tr_loss=0.8028 val_loss=0.7118 F1=0.2900 AUPRC=0.1125 ROC_AUC=0.6563 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep006 | tr_loss=0.8156 val_loss=0.9374 F1=0.0751 AUPRC=0.0686 ROC_AUC=0.5171 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep007 | tr_loss=0.8449 val_loss=0.7221 F1=0.2778 AUPRC=0.1456 ROC_AUC=0.5866 th
r=1.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep008 | tr_loss=0.7364 val_loss=0.8060 F1=0.1942 AUPRC=0.1260 ROC_AUC=0.5545 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep009 | tr_loss=0.8361 val_loss=0.7756 F1=0.2246 AUPRC=0.1378 ROC_AUC=0.5647 th
r=0.000 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep010 | tr_loss=0.7694 val_loss=0.7509 F1=0.2491 AUPRC=0.1444 ROC_AUC=0.5738 th
r=0.382 lr=1.00e-03 | best_AUPRC=0.3253 (ep1)
ep011 | tr_loss=0.7699 val_loss=0.7522 F1=0.2485 AUPRC=0.1450 ROC_AUC=0.5735 th
r=0.000 lr=5.00e-04 | best_AUPRC=0.3253 (ep1)
■ early stop @ ep11 (no improv 10 epochs)

```

advanced | DiceLoss



```

weights -> best_mlp_advanced_DiceLoss.pth
history -> history_advanced_DiceLoss.json
VAL : {'F1': 0.36814658210007045, 'Precision': 0.5257648953301127, 'Recall':
0.2832357406202559, 'ROC_AUC': 0.7461929480777164, 'AUPRC': 0.3252653120297071}
TEST : {'F1': 0.2971898197242842, 'Precision': 0.3221264367816092, 'Recall': 0.
27583661417322836, 'ROC_AUC': 0.739077256573095, 'AUPRC': 0.24916496071867877} (t
hr=0.0000)

```

Out[11]:

	run_id	model	bottleneck	dropout	loss_fn	ep
--	--------	-------	------------	---------	---------	----

0	13	FraudMLP_simple	-	0.15	BCEWithLogitsLoss+pos_weight	
---	----	-----------------	---	------	------------------------------	--

1	14	FraudMLP_simple	-	0.15	FocalLoss(alpha=0.75,gamma=2.0)	
---	----	-----------------	---	------	---------------------------------	--

6	19	FraudMLP_advanced	-	0.15	BCEWithLogitsLoss+pos_weight	
---	----	-------------------	---	------	------------------------------	--

7	20	FraudMLP_advanced	-	0.15	FocalLoss(alpha=0.75,gamma=2.0)	
---	----	-------------------	---	------	---------------------------------	--

3	16	FraudMLP_medium	-	0.15	BCEWithLogitsLoss+pos_weight	
---	----	-----------------	---	------	------------------------------	--

4	17	FraudMLP_medium	-	0.15	FocalLoss(alpha=0.75,gamma=2.0)	
---	----	-----------------	---	------	---------------------------------	--

2	15	FraudMLP_simple	-	0.15		DiceLoss
---	----	-----------------	---	------	--	----------

5	18	FraudMLP_medium	-	0.15		DiceLoss
---	----	-----------------	---	------	--	----------

run_id	model	bottleneck	dropout	loss_fn	ep
8	21	FraudMLP_advanced	-	0.15	DiceLoss

Wnioski z eksperymentu:

- Funkcja straty DiceLoss nie radzi sobie za dobrze
- Funkcje BCE i Focal Loss bardzo podobnie sobie radziły
- Najlepiej na Test AUC i AUPRC wypadła kombinacja medium z Focal Loss
- Aktualnie modele były trenowane z $lr = e-3$, należy zmniejszyć learning rate
- Działaliśmy na stosunkowo dużym batch size, zmienimy to w dalszym tune modelu

W dalszej analizie skupimy się na architekturze Medium + Focal loss, zmniejszymy learning rate

```
In [12]: # Run 10: Lr=5e-4, dropout=0.30
summary_10 = run_grid_search(
    X_train_proc, y_train, X_valid_proc, y_valid, X_test_proc, y_test,
    architectures=("medium",),
    losses=("FocalLoss(alpha=0.75,gamma=2.0)",),
    max_epochs=100, patience=15, es_metric="AUPRC", pos_weight_cap=10.0,
    lr=5e-4, batch_size=512,
    dropout=0.15, tag_suffix="_run10",
)

# Run 11: Lr=5e-4, dropout=0.30
summary_11 = run_grid_search(
    X_train_proc, y_train, X_valid_proc, y_valid, X_test_proc, y_test,
    architectures=("medium",),
    losses=("FocalLoss(alpha=0.75,gamma=2.0)",),
    max_epochs=100, patience=15, es_metric="AUPRC", pos_weight_cap=10.0,
    lr=5e-4, batch_size=512,
    dropout=0.30, tag_suffix="_run11",
)

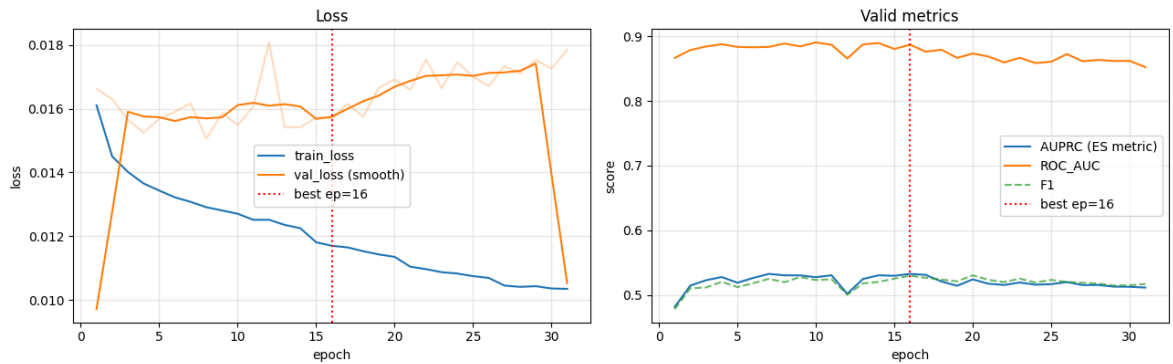
# Run 12: Lr=1e-3, dropout=0.30 (finalny)
summary_12 = run_grid_search(
    X_train_proc, y_train, X_valid_proc, y_valid, X_test_proc, y_test,
    architectures=("medium",),
    losses=("FocalLoss(alpha=0.75,gamma=2.0)",),
    max_epochs=100, patience=15, es_metric="AUPRC", pos_weight_cap=10.0,
    lr=1e-3, batch_size=512,
    dropout=0.30, tag_suffix="_run12",
)
```

pos_weight raw=28.56, used=10.00 (cap=10.0); train_pos_rate=0.0338

```
===== MEDIUM | FocalLoss(alpha=0.75,gamma=2.0) | dropout=0.15 =====
ep001 | tr_loss=0.0161 val_loss=0.0166 F1=0.4781 AUPRC=0.4814 ROC_AUC=0.8667 th
r=0.421 lr=5.00e-04 | best_AUPRC=0.4814 (ep1)
ep002 | tr_loss=0.0145 val_loss=0.0163 F1=0.5102 AUPRC=0.5145 ROC_AUC=0.8789 th
r=0.501 lr=5.00e-04 | best_AUPRC=0.5145 (ep2)
ep003 | tr_loss=0.0140 val_loss=0.0157 F1=0.5115 AUPRC=0.5227 ROC_AUC=0.8844 th
r=0.413 lr=5.00e-04 | best_AUPRC=0.5227 (ep3)
ep004 | tr_loss=0.0137 val_loss=0.0152 F1=0.5203 AUPRC=0.5277 ROC_AUC=0.8879 th
r=0.457 lr=5.00e-04 | best_AUPRC=0.5277 (ep4)
ep005 | tr_loss=0.0134 val_loss=0.0157 F1=0.5120 AUPRC=0.5188 ROC_AUC=0.8837 th
r=0.501 lr=5.00e-04 | best_AUPRC=0.5277 (ep4)
ep006 | tr_loss=0.0132 val_loss=0.0159 F1=0.5181 AUPRC=0.5262 ROC_AUC=0.8832 th
r=0.424 lr=5.00e-04 | best_AUPRC=0.5277 (ep4)
ep007 | tr_loss=0.0131 val_loss=0.0162 F1=0.5248 AUPRC=0.5325 ROC_AUC=0.8837 th
r=0.435 lr=5.00e-04 | best_AUPRC=0.5325 (ep7)
ep008 | tr_loss=0.0129 val_loss=0.0151 F1=0.5197 AUPRC=0.5302 ROC_AUC=0.8889 th
r=0.479 lr=5.00e-04 | best_AUPRC=0.5325 (ep7)
ep009 | tr_loss=0.0128 val_loss=0.0159 F1=0.5277 AUPRC=0.5302 ROC_AUC=0.8846 th
r=0.446 lr=5.00e-04 | best_AUPRC=0.5325 (ep7)
ep010 | tr_loss=0.0127 val_loss=0.0155 F1=0.5230 AUPRC=0.5274 ROC_AUC=0.8907 th
r=0.449 lr=5.00e-04 | best_AUPRC=0.5325 (ep7)
ep011 | tr_loss=0.0125 val_loss=0.0161 F1=0.5242 AUPRC=0.5303 ROC_AUC=0.8870 th
r=0.428 lr=5.00e-04 | best_AUPRC=0.5325 (ep7)
ep012 | tr_loss=0.0125 val_loss=0.0181 F1=0.5003 AUPRC=0.5017 ROC_AUC=0.8658 th
r=0.427 lr=5.00e-04 | best_AUPRC=0.5325 (ep7)
ep013 | tr_loss=0.0124 val_loss=0.0154 F1=0.5178 AUPRC=0.5246 ROC_AUC=0.8876 th
r=0.467 lr=5.00e-04 | best_AUPRC=0.5325 (ep7)
ep014 | tr_loss=0.0122 val_loss=0.0154 F1=0.5201 AUPRC=0.5303 ROC_AUC=0.8897 th
r=0.507 lr=2.50e-04 | best_AUPRC=0.5325 (ep7)
ep015 | tr_loss=0.0118 val_loss=0.0157 F1=0.5251 AUPRC=0.5296 ROC_AUC=0.8805 th
r=0.511 lr=2.50e-04 | best_AUPRC=0.5325 (ep7)
ep016 | tr_loss=0.0117 val_loss=0.0157 F1=0.5296 AUPRC=0.5325 ROC_AUC=0.8871 th
r=0.446 lr=2.50e-04 | best_AUPRC=0.5325 (ep16)
ep017 | tr_loss=0.0117 val_loss=0.0162 F1=0.5265 AUPRC=0.5310 ROC_AUC=0.8764 th
r=0.441 lr=2.50e-04 | best_AUPRC=0.5325 (ep16)
ep018 | tr_loss=0.0115 val_loss=0.0157 F1=0.5237 AUPRC=0.5208 ROC_AUC=0.8794 th
r=0.476 lr=2.50e-04 | best_AUPRC=0.5325 (ep16)
ep019 | tr_loss=0.0114 val_loss=0.0167 F1=0.5210 AUPRC=0.5141 ROC_AUC=0.8667 th
r=0.445 lr=2.50e-04 | best_AUPRC=0.5325 (ep16)
ep020 | tr_loss=0.0114 val_loss=0.0169 F1=0.5303 AUPRC=0.5240 ROC_AUC=0.8735 th
r=0.441 lr=1.25e-04 | best_AUPRC=0.5325 (ep16)
ep021 | tr_loss=0.0110 val_loss=0.0166 F1=0.5232 AUPRC=0.5173 ROC_AUC=0.8689 th
r=0.442 lr=1.25e-04 | best_AUPRC=0.5325 (ep16)
ep022 | tr_loss=0.0110 val_loss=0.0175 F1=0.5199 AUPRC=0.5154 ROC_AUC=0.8597 th
r=0.441 lr=1.25e-04 | best_AUPRC=0.5325 (ep16)
ep023 | tr_loss=0.0109 val_loss=0.0167 F1=0.5251 AUPRC=0.5192 ROC_AUC=0.8669 th
r=0.462 lr=1.25e-04 | best_AUPRC=0.5325 (ep16)
ep024 | tr_loss=0.0108 val_loss=0.0175 F1=0.5192 AUPRC=0.5158 ROC_AUC=0.8588 th
r=0.461 lr=1.25e-04 | best_AUPRC=0.5325 (ep16)
ep025 | tr_loss=0.0107 val_loss=0.0170 F1=0.5232 AUPRC=0.5166 ROC_AUC=0.8608 th
r=0.443 lr=1.25e-04 | best_AUPRC=0.5325 (ep16)
ep026 | tr_loss=0.0107 val_loss=0.0167 F1=0.5200 AUPRC=0.5199 ROC_AUC=0.8727 th
r=0.445 lr=6.25e-05 | best_AUPRC=0.5325 (ep16)
ep027 | tr_loss=0.0105 val_loss=0.0173 F1=0.5187 AUPRC=0.5152 ROC_AUC=0.8615 th
r=0.499 lr=6.25e-05 | best_AUPRC=0.5325 (ep16)
ep028 | tr_loss=0.0104 val_loss=0.0171 F1=0.5175 AUPRC=0.5153 ROC_AUC=0.8636 th
r=0.453 lr=6.25e-05 | best_AUPRC=0.5325 (ep16)
ep029 | tr_loss=0.0104 val_loss=0.0175 F1=0.5145 AUPRC=0.5128 ROC_AUC=0.8619 th
```

r=0.448 lr=6.25e-05 | best_AUPRC=0.5325 (ep16)
ep030 | tr_loss=0.0104 val_loss=0.0173 F1=0.5148 AUPRC=0.5125 ROC_AUC=0.8621 th
r=0.469 lr=6.25e-05 | best_AUPRC=0.5325 (ep16)
ep031 | tr_loss=0.0104 val_loss=0.0178 F1=0.5169 AUPRC=0.5113 ROC_AUC=0.8525 th
r=0.522 lr=6.25e-05 | best_AUPRC=0.5325 (ep16)
■ early stop @ ep31 (no improv 15 epochs)

medium | FocalLoss(alpha=0.75,gamma=2.0)



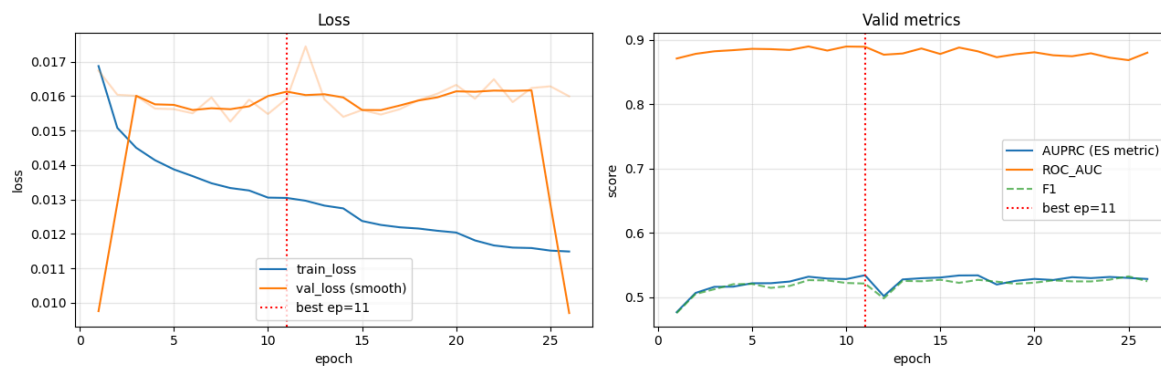

```
weights -> best_mlp_medium_FocalLoss_alpha075_gamma20_run10.pth
history -> history_medium_FocalLoss_alpha075_gamma20_run10.json
VAL : {'F1': 0.5296100144439095, 'Precision': 0.595077089532053, 'Recall': 0.4
771199306007374, 'ROC_AUC': 0.8871317892265677, 'AUPRC': 0.5325230735759089}
TEST : {'F1': 0.43479384774460383, 'Precision': 0.4579362918595154, 'Recall':
0.4138779527559055, 'ROC_AUC': 0.8416539120002585, 'AUPRC': 0.396318606621589} (t
hr=0.4459)
pos_weight raw=28.56, used=10.00 (cap=10.0); train_pos_rate=0.0338
```

```
===== MEDIUM | FocalLoss(alpha=0.75,gamma=2.0) | dropout=0.3 =====
ep001 | tr_loss=0.0169 val_loss=0.0167 F1=0.4756 AUPRC=0.4769 ROC_AUC=0.8709 th
r=0.408 lr=5.00e-04 | best_AUPRC=0.4769 (ep1)
ep002 | tr_loss=0.0151 val_loss=0.0160 F1=0.5051 AUPRC=0.5068 ROC_AUC=0.8782 th
r=0.428 lr=5.00e-04 | best_AUPRC=0.5068 (ep2)
ep003 | tr_loss=0.0145 val_loss=0.0160 F1=0.5126 AUPRC=0.5163 ROC_AUC=0.8821 th
r=0.412 lr=5.00e-04 | best_AUPRC=0.5163 (ep3)
ep004 | tr_loss=0.0141 val_loss=0.0156 F1=0.5204 AUPRC=0.5165 ROC_AUC=0.8839 th
r=0.452 lr=5.00e-04 | best_AUPRC=0.5165 (ep4)
ep005 | tr_loss=0.0139 val_loss=0.0156 F1=0.5208 AUPRC=0.5218 ROC_AUC=0.8860 th
r=0.456 lr=5.00e-04 | best_AUPRC=0.5218 (ep5)
ep006 | tr_loss=0.0137 val_loss=0.0155 F1=0.5147 AUPRC=0.5219 ROC_AUC=0.8855 th
r=0.449 lr=5.00e-04 | best_AUPRC=0.5219 (ep6)
ep007 | tr_loss=0.0135 val_loss=0.0160 F1=0.5177 AUPRC=0.5245 ROC_AUC=0.8842 th
r=0.432 lr=5.00e-04 | best_AUPRC=0.5245 (ep7)
ep008 | tr_loss=0.0133 val_loss=0.0153 F1=0.5267 AUPRC=0.5320 ROC_AUC=0.8897 th
r=0.477 lr=5.00e-04 | best_AUPRC=0.5320 (ep8)
ep009 | tr_loss=0.0133 val_loss=0.0159 F1=0.5262 AUPRC=0.5291 ROC_AUC=0.8832 th
r=0.437 lr=5.00e-04 | best_AUPRC=0.5320 (ep8)
ep010 | tr_loss=0.0131 val_loss=0.0155 F1=0.5223 AUPRC=0.5283 ROC_AUC=0.8895 th
r=0.454 lr=5.00e-04 | best_AUPRC=0.5320 (ep8)
ep011 | tr_loss=0.0130 val_loss=0.0159 F1=0.5214 AUPRC=0.5343 ROC_AUC=0.8892 th
r=0.432 lr=5.00e-04 | best_AUPRC=0.5343 (ep11)
ep012 | tr_loss=0.0130 val_loss=0.0174 F1=0.4983 AUPRC=0.5019 ROC_AUC=0.8769 th
r=0.418 lr=5.00e-04 | best_AUPRC=0.5343 (ep11)
ep013 | tr_loss=0.0128 val_loss=0.0159 F1=0.5255 AUPRC=0.5277 ROC_AUC=0.8787 th
r=0.472 lr=5.00e-04 | best_AUPRC=0.5343 (ep11)
ep014 | tr_loss=0.0127 val_loss=0.0154 F1=0.5249 AUPRC=0.5297 ROC_AUC=0.8865 th
r=0.558 lr=2.50e-04 | best_AUPRC=0.5343 (ep11)
ep015 | tr_loss=0.0124 val_loss=0.0156 F1=0.5272 AUPRC=0.5307 ROC_AUC=0.8780 th
r=0.497 lr=2.50e-04 | best_AUPRC=0.5343 (ep11)
ep016 | tr_loss=0.0123 val_loss=0.0155 F1=0.5225 AUPRC=0.5339 ROC_AUC=0.8880 th
r=0.443 lr=2.50e-04 | best_AUPRC=0.5343 (ep11)
ep017 | tr_loss=0.0122 val_loss=0.0156 F1=0.5271 AUPRC=0.5341 ROC_AUC=0.8821 th
r=0.458 lr=2.50e-04 | best_AUPRC=0.5343 (ep11)
ep018 | tr_loss=0.0122 val_loss=0.0159 F1=0.5243 AUPRC=0.5199 ROC_AUC=0.8729 th
r=0.492 lr=2.50e-04 | best_AUPRC=0.5343 (ep11)
ep019 | tr_loss=0.0121 val_loss=0.0161 F1=0.5210 AUPRC=0.5256 ROC_AUC=0.8775 th
r=0.455 lr=2.50e-04 | best_AUPRC=0.5343 (ep11)
ep020 | tr_loss=0.0120 val_loss=0.0163 F1=0.5227 AUPRC=0.5285 ROC_AUC=0.8806 th
r=0.450 lr=1.25e-04 | best_AUPRC=0.5343 (ep11)
ep021 | tr_loss=0.0118 val_loss=0.0159 F1=0.5263 AUPRC=0.5269 ROC_AUC=0.8759 th
r=0.456 lr=1.25e-04 | best_AUPRC=0.5343 (ep11)
ep022 | tr_loss=0.0117 val_loss=0.0165 F1=0.5248 AUPRC=0.5313 ROC_AUC=0.8744 th
r=0.430 lr=1.25e-04 | best_AUPRC=0.5343 (ep11)
ep023 | tr_loss=0.0116 val_loss=0.0158 F1=0.5247 AUPRC=0.5298 ROC_AUC=0.8790 th
r=0.469 lr=1.25e-04 | best_AUPRC=0.5343 (ep11)
ep024 | tr_loss=0.0116 val_loss=0.0162 F1=0.5274 AUPRC=0.5317 ROC_AUC=0.8722 th
r=0.446 lr=1.25e-04 | best_AUPRC=0.5343 (ep11)
ep025 | tr_loss=0.0115 val_loss=0.0163 F1=0.5325 AUPRC=0.5301 ROC_AUC=0.8684 th
r=0.447 lr=1.25e-04 | best_AUPRC=0.5343 (ep11)
```

ep026 | tr_loss=0.0115 val_loss=0.0160 F1=0.5250 AUPRC=0.5285 ROC_AUC=0.8800 th
r=0.458 lr=6.25e-05 | best_AUPRC=0.5343 (ep11)

■ early stop @ ep26 (no improv 15 epochs)

medium | FocalLoss(alpha=0.75,gamma=2.0)



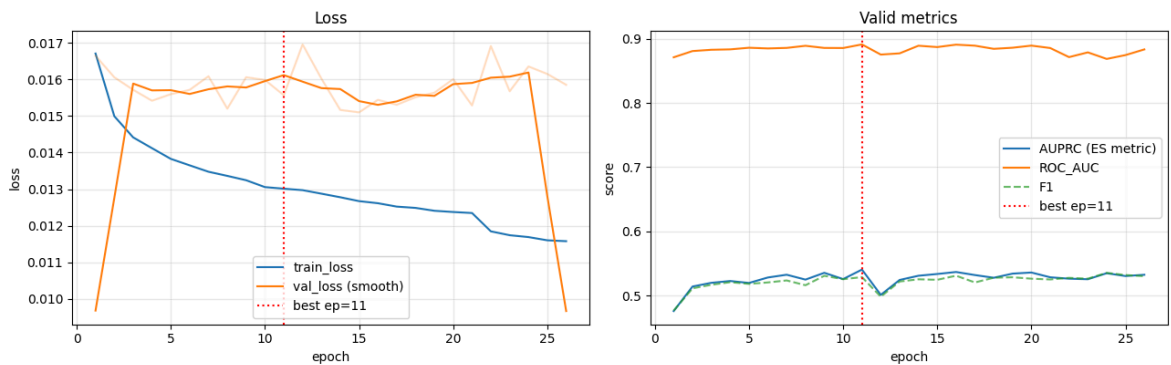
```
weights -> best_mlp_medium_FocalLoss_alpha075_gamma20_run11.pth
history -> history_medium_FocalLoss_alpha075_gamma20_run11.json
VAL : {'F1': 0.521399798590131, 'Precision': 0.6213621362136214, 'Recall': 0.4
4914335285187595, 'ROC_AUC': 0.8891855075597352, 'AUPRC': 0.534270456511618}
TEST : {'F1': 0.45508486270180765, 'Precision': 0.5180647188187245, 'Recall':
0.40575787401574803, 'ROC_AUC': 0.8591438860401857, 'AUPRC': 0.4111309364686261}
(thr=0.4315)
pos_weight raw=28.56, used=10.00 (cap=10.0); train_pos_rate=0.0338
```

```
===== MEDIUM | FocalLoss(alpha=0.75,gamma=2.0) | dropout=0.3 =====
ep001 | tr_loss=0.0167 val_loss=0.0166 F1=0.4763 AUPRC=0.4757 ROC_AUC=0.8712 th
r=0.415 lr=1.00e-03 | best_AUPRC=0.4757 (ep1)
ep002 | tr_loss=0.0150 val_loss=0.0161 F1=0.5111 AUPRC=0.5139 ROC_AUC=0.8810 th
r=0.448 lr=1.00e-03 | best_AUPRC=0.5139 (ep2)
ep003 | tr_loss=0.0144 val_loss=0.0157 F1=0.5166 AUPRC=0.5197 ROC_AUC=0.8830 th
r=0.430 lr=1.00e-03 | best_AUPRC=0.5197 (ep3)
ep004 | tr_loss=0.0141 val_loss=0.0154 F1=0.5207 AUPRC=0.5225 ROC_AUC=0.8836 th
r=0.469 lr=1.00e-03 | best_AUPRC=0.5225 (ep4)
ep005 | tr_loss=0.0138 val_loss=0.0156 F1=0.5180 AUPRC=0.5195 ROC_AUC=0.8861 th
r=0.452 lr=1.00e-03 | best_AUPRC=0.5225 (ep4)
ep006 | tr_loss=0.0137 val_loss=0.0157 F1=0.5203 AUPRC=0.5280 ROC_AUC=0.8851 th
r=0.427 lr=1.00e-03 | best_AUPRC=0.5280 (ep6)
ep007 | tr_loss=0.0135 val_loss=0.0161 F1=0.5234 AUPRC=0.5323 ROC_AUC=0.8859 th
r=0.440 lr=1.00e-03 | best_AUPRC=0.5323 (ep7)
ep008 | tr_loss=0.0134 val_loss=0.0152 F1=0.5159 AUPRC=0.5247 ROC_AUC=0.8892 th
r=0.487 lr=1.00e-03 | best_AUPRC=0.5323 (ep7)
ep009 | tr_loss=0.0132 val_loss=0.0161 F1=0.5306 AUPRC=0.5353 ROC_AUC=0.8858 th
r=0.416 lr=1.00e-03 | best_AUPRC=0.5353 (ep9)
ep010 | tr_loss=0.0131 val_loss=0.0160 F1=0.5253 AUPRC=0.5255 ROC_AUC=0.8857 th
r=0.414 lr=1.00e-03 | best_AUPRC=0.5353 (ep9)
ep011 | tr_loss=0.0130 val_loss=0.0156 F1=0.5286 AUPRC=0.5403 ROC_AUC=0.8913 th
r=0.474 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep012 | tr_loss=0.0130 val_loss=0.0170 F1=0.4981 AUPRC=0.5009 ROC_AUC=0.8755 th
r=0.425 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep013 | tr_loss=0.0129 val_loss=0.0160 F1=0.5217 AUPRC=0.5241 ROC_AUC=0.8774 th
r=0.435 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep014 | tr_loss=0.0128 val_loss=0.0152 F1=0.5252 AUPRC=0.5307 ROC_AUC=0.8893 th
r=0.513 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep015 | tr_loss=0.0127 val_loss=0.0151 F1=0.5244 AUPRC=0.5335 ROC_AUC=0.8872 th
r=0.494 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep016 | tr_loss=0.0126 val_loss=0.0154 F1=0.5308 AUPRC=0.5366 ROC_AUC=0.8910 th
r=0.430 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep017 | tr_loss=0.0125 val_loss=0.0153 F1=0.5203 AUPRC=0.5318 ROC_AUC=0.8894 th
r=0.481 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep018 | tr_loss=0.0125 val_loss=0.0155 F1=0.5276 AUPRC=0.5276 ROC_AUC=0.8844 th
r=0.481 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep019 | tr_loss=0.0124 val_loss=0.0156 F1=0.5283 AUPRC=0.5340 ROC_AUC=0.8862 th
r=0.430 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep020 | tr_loss=0.0124 val_loss=0.0160 F1=0.5261 AUPRC=0.5358 ROC_AUC=0.8894 th
r=0.463 lr=1.00e-03 | best_AUPRC=0.5403 (ep11)
ep021 | tr_loss=0.0123 val_loss=0.0153 F1=0.5249 AUPRC=0.5283 ROC_AUC=0.8857 th
r=0.465 lr=5.00e-04 | best_AUPRC=0.5403 (ep11)
ep022 | tr_loss=0.0118 val_loss=0.0169 F1=0.5274 AUPRC=0.5261 ROC_AUC=0.8715 th
r=0.401 lr=5.00e-04 | best_AUPRC=0.5403 (ep11)
ep023 | tr_loss=0.0117 val_loss=0.0157 F1=0.5261 AUPRC=0.5253 ROC_AUC=0.8789 th
r=0.485 lr=5.00e-04 | best_AUPRC=0.5403 (ep11)
ep024 | tr_loss=0.0117 val_loss=0.0164 F1=0.5352 AUPRC=0.5347 ROC_AUC=0.8687 th
r=0.468 lr=5.00e-04 | best_AUPRC=0.5403 (ep11)
ep025 | tr_loss=0.0116 val_loss=0.0161 F1=0.5318 AUPRC=0.5302 ROC_AUC=0.8747 th
r=0.443 lr=5.00e-04 | best_AUPRC=0.5403 (ep11)
```

ep026 | tr_loss=0.0116 val_loss=0.0159 F1=0.5300 AUPRC=0.5322 ROC_AUC=0.8835 th
r=0.459 lr=5.00e-04 | best_AUPRC=0.5403 (ep11)

■ early stop @ ep26 (no improv 15 epochs)

medium | FocalLoss(alpha=0.75,gamma=2.0)



weights -> best_mlp_medium_FocalLoss_alpha075_gamma20_run12.pth
history -> history_medium_FocalLoss_alpha075_gamma20_run12.json
VAL : {'F1': 0.5285623003194888, 'Precision': 0.6434349719975109, 'Recall': 0.44849273476469315, 'ROC_AUC': 0.8913060855951283, 'AUPRC': 0.5403103068287689}
TEST : {'F1': 0.44830972085846543, 'Precision': 0.5213703099510604, 'Recall': 0.39320866141732286, 'ROC_AUC': 0.8626313247190545, 'AUPRC': 0.41979508530324783}
(thr=0.4741)

Kody generujące tabele i wykresy do raportu

```
In [13]: # Wykresy ewaluacyjne modelu finalnego (Run 12: Medium + Focal Loss)
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
    precision_score, recall_score,
)
from matplotlib.gridspec import GridSpec

# 1. Wczytanie modelu finalnego z pliku
# Run 12: lr=1e-3, bs=512, dropout=0.30, patience=15
# Ostrożnie: `model` w pamięci = Run 10 (lr=5e-4) z poprzedniej komórki
FINAL_WEIGHTS = ART_DIR / "best_mlp_medium_FocalLoss_alpha075_gamma20_run12.pth"

final_model = MediumMLP(in_dim=X_test_proc.shape[1], dropout=0.30).to(DEVICE)
final_model.load_state_dict(torch.load(FINAL_WEIGHTS, map_location=DEVICE))
final_model.eval()

# 2. Predykcje i próg decyzyjny
val_proba_f = predict_proba(final_model, X_valid_proc)
test_proba_f = predict_proba(final_model, X_test_proc)

# Próg wyznaczony na zbiorze walidacyjnym (max F1)
thr_f, _ = best_f1_threshold(y_valid.values, val_proba_f)

y_test_arr = y_test.values
y_pred_test = (test_proba_f >= thr_f).astype(int)

print(f"Próg decyzyjny (max F1 na valid) : {thr_f:.4f}")
print("Metryki testowe :", evaluate(y_test_arr, test_proba_f, thr_f))

# 3. Figura: PR + CM na górze, rozkład scorów na dole
fig = plt.figure(figsize=(12, 9))
fig.suptitle(
    "Model finalny: Medium MLP + Focal Loss (α=0.75, γ=2.0) | Zbiór testowy 00
```

```

        fontsize=12, fontweight="bold",
    )

gs = GridSpec(2, 2, figure=fig, hspace=0.38, wspace=0.30)
ax_pr = fig.add_subplot(gs[0, 0]) # góra lewo - Precision-Recall
ax_cm = fig.add_subplot(gs[0, 1]) # góra prawo - Macierz pomyłek
ax_sc = fig.add_subplot(gs[1, :]) # dół całość - Rozkład scorów

# 3a. Krzywa Precision-Recall
prec_c, rec_c, _ = precision_recall_curve(y_test_arr, test_proba_f)
auprc = average_precision_score(y_test_arr, test_proba_f)
op_prec = precision_score(y_test_arr, y_pred_test, zero_division=0)
op_rec = recall_score(y_test_arr, y_pred_test, zero_division=0)

ax_pr.plot(rec_c, prec_c, color="#2C5784", lw=2, label=f"AUPRC = {auprc:.4f}")
ax_pr.axhline(
    y_test_arr.mean(), color="gray", linestyle="--", lw=1.2,
    label=f"Baseline (losowy) = {y_test_arr.mean():.4f}",
)
ax_pr.scatter(
    op_rec, op_prec, color="#A63D40", s=80, zorder=5,
    label=f"Próg = {thr_f:.2f}\n(P={op_prec:.2f}, R={op_rec:.2f})",
)
ax_pr.set_xlabel("Recall")
ax_pr.set_ylabel("Precision")
ax_pr.set_title("Krzywa Precision-Recall")
ax_pr.legend(fontsize=9)
ax_pr.grid(alpha=0.3)

# 3b. Macierz pomyłek
cm_raw = confusion_matrix(y_test_arr, y_pred_test)
cm_norm = confusion_matrix(y_test_arr, y_pred_test, normalize="true")
tn, fp, fn, tp = cm_raw.ravel()

disp = ConfusionMatrixDisplay(cm_norm, display_labels=["Legit", "Fraud"])
disp.plot(ax=ax_cm, cmap="Blues", values_format=".2%", colorbar=False)

for (i, j), val in np.ndenumerate(cm_raw):
    ax_cm.text(j, i + 0.25, f"n={val:,.2f}", ha="center", va="center",
               fontsize=9, color="gray")

ax_cm.set_title(f"Macierz pomyłek (próg = {thr_f:.2f})")

# 3c. Rozkład scorów
ax_sc.hist(
    test_proba_f[y_test_arr == 0], bins=80, density=True,
    alpha=0.6, color="#5B8DB8", label="Legit (klasa 0)",
)
ax_sc.hist(
    test_proba_f[y_test_arr == 1], bins=80, density=True,
    alpha=0.6, color="#A63D40", label="Fraud (klasa 1)",
)
ax_sc.axvline(
    thr_f, color="black", linestyle="--", lw=1.5,
    label=f"Próg = {thr_f:.2f}",
)
ax_sc.set_xlabel("P(isFraud)", fontsize=11)
ax_sc.set_ylabel("Gęstość", fontsize=11)
ax_sc.set_title("Rozkład scorów")
ax_sc.legend(fontsize=10)

```

```

ax_sc.grid(alpha=0.3)

# 4. Zapis
save_path = PLOT_DIR / "final_model_evaluation.png"
fig.savefig(save_path, dpi=150, bbox_inches="tight")
plt.show()
print(f"\nZapisano: {save_path}")

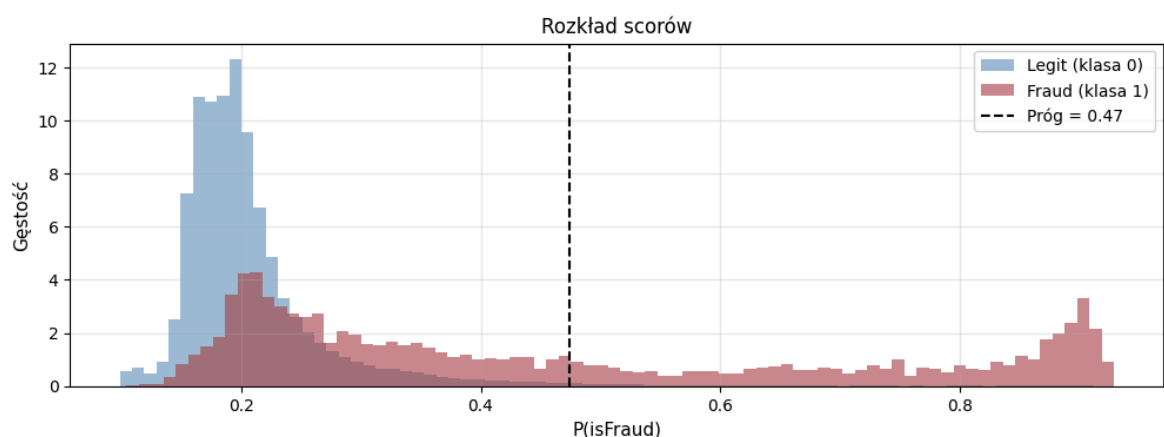
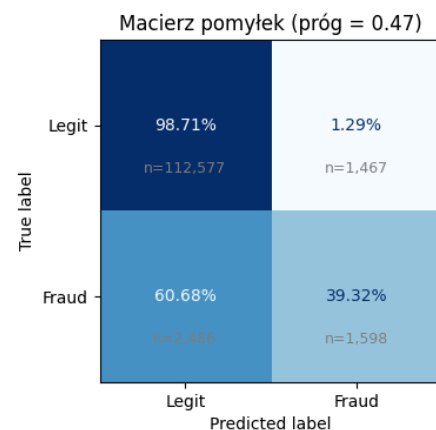
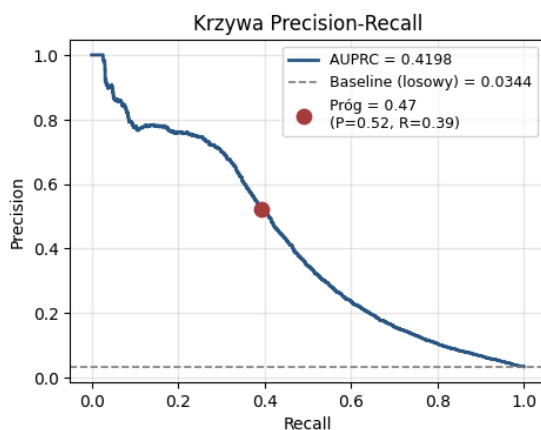
# 5. Podsumowanie macierzy pomyłek
print(f"\n{'-'*45}")
print(f"   TPR / Recall      : {tp/(tp+fn):.2%} | FNR (przeoczone fraudy) : {fn/"}
print(f"   TNR               : {tn/(tn+fp):.2%} | FPR (fałszywy alarm)   : {fp/"}
print(f"   Precision         : {tp/(tp+fp):.2%}")
print(f"\n{'-'*45}")

```

Próg decyzyjny (max F1 na valid) : 0.4741

Metryki testowe : {'F1': 0.44830972085846543, 'Precision': 0.5213703099510604, 'Recall': 0.39320866141732286, 'ROC_AUC': 0.8626313247190545, 'AUPRC': 0.41979508530324783}

Model finalny: Medium MLP + Focal Loss ($\alpha=0.75$, $\gamma=2.0$) | Zbiór testowy OOT



Zapisano: artifacts_mlp\plots\final_model_evaluation.png

```

-----
TPR / Recall      : 39.32% | FNR (przeoczone fraudy) : 60.68%
TNR               : 98.71% | FPR (fałszywy alarm)   : 1.29%
Precision         : 52.14%
-----

```

**Tabela ewaluacyjna modeli z zapisanych wag
(Train/Val/Test AUPRC + AUC)**

In [14]: EVAL_XLSX = Path("wyniki_eksperymentów.xlsx")

```
def _tag_safe(arch: str, loss_name: str) -> str:
    """Ta sama sanitizacja nazw co w run_grid_search."""
    tag = f"{arch}_{loss_name}".replace(" ", "").replace("/", "")
    return (tag
            .replace("+", "_")
            .replace("(", "_").replace(")", "")
            .replace("=", "").replace(",", "_").replace(".", ""))

def _auprc_auc(model: nn.Module, X: np.ndarray, y: np.ndarray) -> tuple[float, float]:
    proba = predict_proba(model, X)  # robi model.eval() + sigmoid pod spodem
    return average_precision_score(y, proba), roc_auc_score(y, proba)

def build_eval_table(
    X_train_proc, y_train, X_valid_proc, y_valid, X_test_proc, y_test,
    specs: list[tuple[str, str, str]] | None = None,
    architectures=("simple", "medium", "advanced"),
    losses=("BCEWithLogitsLoss+pos_weight",
            "FocalLoss(alpha=0.75,gamma=2.0)",
            "DiceLoss"),
    tag_suffix: str = "",
    art_dir: Path = ART_DIR,
    hist_dir: Path = HIST_DIR,
    save_path: Path = EVAL_XLSX,
    es_metric: str = "AUPRC",
) -> pd.DataFrame:
    """Tabela ewaluacyjna z zapisanych wag, z DWIEMA deltami.

    specs : opcjonalna lista krotek (arch, loss_name, nazwa_pliku_wag). Przy N
            iteruje po (architectures x losses) + tag_suffix i bierze pliki, k
            istnieją na dysku.

    delta (Train-Val) = przeuczenie; delta (Val-Test) = spadek na OOT.
    Dropout nieistotny przy ewaluacji (model.eval() go wyłącza).
    """
    in_dim = X_train_proc.shape[1]
    ytr = np.asarray(y_train); yva = np.asarray(y_valid); yte = np.asarray(y_test)

    if specs is None:
        specs = [(a, l, f"best_mlp_{_tag_safe(a, l)}{tag_suffix}.pth")
                 for a in architectures for l in losses]

    rows = []
    for arch, loss_name, wfname in specs:
        wpath = art_dir / wfname
        if not wpath.exists():
            print(f"  pomijam (brak wag): {wfname}")
            continue

        model = build_model(arch, in_dim).to(DEVICE)  # dropout nieistotny w ev
        model.load_state_dict(torch.load(wpath, map_location=DEVICE))
        model.eval()

        tr_ap, tr_auc = _auprc_auc(model, X_train_proc, ytr)
        va_ap, va_auc = _auprc_auc(model, X_valid_proc, yva)
```

```

te_ap, te_auc = _auprc_auc(model, X_test_proc, yte)

# Najlepsza epoka: argmax val_AUPRC z odpowiadającej historii.
hist_name = "history_" + wfname[len("best_mlp_")].replace(".pth", ".json")
hpath = hist_dir / hist_name
best_ep = None
if hpath.exists():
    with open(hpath, encoding="utf-8") as f:
        hist = json.load(f)
        key = f"val_{es_metric}"
        if hist.get(key):
            k = int(np.argmax(hist[key]))
            best_ep = hist["epoch"][k]
else:
    print(f" brak historii ({hist_name}) -> 'Najlepsza epoka' = NaN dla

rows.append({
    "Architektura": arch,
    "Fn straty": loss_name,
    "Train AUPRC": round(tr_ap, 4),
    "Train AUC": round(tr_auc, 4),
    "Val AUPRC": round(va_ap, 4),
    "Val AUC": round(va_auc, 4),
    "Test AUPRC": round(te_ap, 4),
    "Test AUC": round(te_auc, 4),
    "Najlepsza epoka": best_ep,
    "delta AUPRC (Train-Val)": round(tr_ap - va_ap, 4), # przeuczenie
    "delta AUC (Train-Val)": round(tr_auc - va_auc, 4), # przeuczenie
    "delta AUPRC (Val-Test)": round(va_ap - te_ap, 4), # generalizacja
    "delta AUC (Val-Test)": round(va_auc - te_auc, 4), # generalizacja
})
print(f" {arch:9s} | {loss_name:35s} | "
      f"Test AUPRC={te_ap:.4f} AUC={te_auc:.4f} | best_ep={best_ep}")

df = pd.DataFrame(rows, columns=[
    "Architektura", "Fn straty", "Train AUPRC", "Train AUC",
    "Val AUPRC", "Val AUC", "Test AUPRC", "Test AUC", "Najlepsza epoka",
    "delta AUPRC (Train-Val)", "delta AUC (Train-Val)",
    "delta AUPRC (Val-Test)", "delta AUC (Val-Test)",
])
df.to_excel(save_path, index=False)
print(f"\nZapisano tabelę ewaluacyjną -> {save_path} ({len(df)} modeli)")
return df

```

```

In [15]: # Tabela ewaluacyjna dla wszystkich modeli z grid searchu (9 plików wag).
# Czyta wagi z artifacts_mlp/, liczy AUPRC + ROC AUC na train/val/test,
# najlepszą epokę z historii i zapisuje do wyniki_eksperymentów.xlsx.
specs = [
    # 9 modeli z grid searchu (selekcja architektury)
    ("simple", "BCEWithLogitsLoss+pos_weight", "best_mlp_simple_BCEWithLogi
    ("simple", "FocalLoss(alpha=0.75,gamma=2.0)", "best_mlp_simple_FocalLoss_a
    ("simple", "DiceLoss", "best_mlp_simple_DiceLoss.pt
    ("medium", "BCEWithLogitsLoss+pos_weight", "best_mlp_medium_BCEWithLogi
    ("medium", "FocalLoss(alpha=0.75,gamma=2.0)", "best_mlp_medium_FocalLoss_a
    ("medium", "DiceLoss", "best_mlp_medium_DiceLoss.pt
    ("advanced", "BCEWithLogitsLoss+pos_weight", "best_mlp_advanced_BCEWithLo
    ("advanced", "FocalLoss(alpha=0.75,gamma=2.0)", "best_mlp_advanced_FocalLoss
    ("advanced", "DiceLoss", "best_mlp_advanced_DiceLoss.
    # 3 modele z tuningu (medium + Focal), rozróżnione etykietą
    ("medium", "Focal - Run 10 (lr=5e-4, bs=512, do=0.15)", "best_mlp_medium_Foc

```



```

    ("medium", "Focal - Run 11 (lr=5e-4, bs=512, do=0.30)", "best_mlp_medium_Foc
    ("medium", "Focal - Run 12 (lr=1e-3, bs=512, do=0.30)", "best_mlp_medium_Foc
]

df_eval = build_eval_table(
    X_train_proc, y_train, X_valid_proc, y_valid, X_test_proc, y_test,
    specs=specs,
    save_path=Path("wyniki_eksperymentów.xlsx"),
)
df_eval

```

simple	BCEWithLogitsLoss+pos_weight	Test AUPRC=0.3977 AUC=0.8600
best_ep=30		
simple	FocalLoss(alpha=0.75,gamma=2.0)	Test AUPRC=0.3946 AUC=0.8595
best_ep=21		
simple	DiceLoss	Test AUPRC=0.3623 AUC=0.7939
best_ep=37		
medium	BCEWithLogitsLoss+pos_weight	Test AUPRC=0.4157 AUC=0.8407
best_ep=17		
medium	FocalLoss(alpha=0.75,gamma=2.0)	Test AUPRC=0.4196 AUC=0.8559
best_ep=7		
medium	DiceLoss	Test AUPRC=0.3563 AUC=0.8053
best_ep=22		
advanced	BCEWithLogitsLoss+pos_weight	Test AUPRC=0.4158 AUC=0.8565
best_ep=5		
advanced	FocalLoss(alpha=0.75,gamma=2.0)	Test AUPRC=0.3967 AUC=0.8580
best_ep=6		
advanced	DiceLoss	Test AUPRC=0.2492 AUC=0.7391
best_ep=1		
medium	Focal - Run 10 (lr=5e-4, bs=512, do=0.15)	Test AUPRC=0.3963 AUC=0.8417
best_ep=16		
medium	Focal - Run 11 (lr=5e-4, bs=512, do=0.30)	Test AUPRC=0.4111 AUC=0.8591
best_ep=11		
medium	Focal - Run 12 (lr=1e-3, bs=512, do=0.30)	Test AUPRC=0.4198 AUC=0.8626
best_ep=11		

Zapisano tabelę ewaluacyjną -> wyniki_eksperymentów.xlsx (12 modeli)

Out[15]:

	Architektura	Fn straty	Train AUPRC	Train AUC	Val AUPRC	Val AUC	T
0	simple	BCEWithLogitsLoss+pos_weight	0.6237	0.9171	0.5442	0.8891	0.39
1	simple	FocalLoss(alpha=0.75,gamma=2.0)	0.6130	0.9098	0.5439	0.8887	0.39
2	simple	DiceLoss	0.4644	0.7776	0.4453	0.8116	0.36
3	medium	BCEWithLogitsLoss+pos_weight	0.6287	0.9166	0.5343	0.8841	0.41
4	medium	FocalLoss(alpha=0.75,gamma=2.0)	0.5938	0.9035	0.5320	0.8868	0.41
5	medium	DiceLoss	0.4005	0.7812	0.4097	0.8160	0.35
6	advanced	BCEWithLogitsLoss+pos_weight	0.6055	0.9058	0.5411	0.8880	0.41
7	advanced	FocalLoss(alpha=0.75,gamma=2.0)	0.6199	0.9070	0.5376	0.8893	0.39
8	advanced	DiceLoss	0.2476	0.7265	0.3253	0.7462	0.24
9	medium	Focal — Run 10 (lr=5e-4, bs=512, do=0.15)	0.6625	0.9225	0.5325	0.8871	0.39
10	medium	Focal — Run 11 (lr=5e-4, bs=512, do=0.30)	0.6130	0.9080	0.5343	0.8892	0.41
11	medium	Focal — Run 12 (lr=1e-3, bs=512, do=0.30)	0.6070	0.9057	0.5403	0.8913	0.41

SCORING MODELU

```
In [16]: def score_dataset(raw_df, model, preprocessor, threshold, *,
                        id_col="TransactionID", target_col=TARGET,
                        positive_label=1, keep_inputs=False):
    """Analog "Oceny punktowej" z SAS EM (faza predykcji / scoring).

    Przyjmuje SUROWE nowe dane (etykieta nieużywana), przepuszcza je przez ten
    sam pipeline preprocessingu co przy treningu i zwraca tabelę scoringową.

    Parametry
    -----
    raw_df      : surowy DataFrame nowych danych (jak `test`/`train` przed FE)
    model       : wytrenowany model (eval mode)
    preprocessor : dopasowany ColumnTransformer (fit tylko na train)
    threshold   : próg decyzyjny dla klasy pozytywnej
    id_col      : kolumna identyfikująca rekord (zachowana w wyniku)
    target_col  : kolumna etykiety - USUWANA, scoring jej nie używa
    keep_inputs : czy dołączyć zmienne wejściowe po inżynierii cech

    Zwraca
    -----
    DataFrame: tabela scoringowa (1 wiersz = 1 zescorowany rekord)
    """
    df = raw_df.copy()
    ids = df[id_col].reset_index(drop=True) if id_col in df.columns else None
```

```

# Faza predykcji: etykieta NIE jest dostępna - jawnie ją odrzucamy,
# by udowodnić, że scoring nie korzysta z labelów (jak "Ocena punktowa").
df = df.drop(columns=[target_col], errors="ignore")

X = engineer_features(df) # ta sama FE co w treningu
X_proc = preprocessor.transform(X).astype(np.float32)

proba1 = predict_proba(model, X_proc) # P(isFraud = 1)
pred = (proba1 >= threshold).astype(int)

out = pd.DataFrame({
    "P_isFraud0": 1.0 - proba1,
    "P_isFraud1": proba1,
    "EM_EVENTPROBABILITY": proba1,
    "EM_CLASSIFICATION": pred,
    "D_isFraud": np.where(pred == positive_label, "Fraud", "Legit")
})
if ids is not None:
    out.insert(0, id_col, ids)
if keep_inputs:
    out = pd.concat([out, X.reset_index(drop=True)], axis=1)
return out

```

```

In [17]: score_input = test.drop(columns=[TARGET]).reset_index(drop=True)
print(f"Zbiór scoringowy: {score_input.shape} "
      f"(kolumna '{TARGET}' usunięta: {TARGET not in score_input.columns})")

# Model finalny (Run 12: Medium MLP + Focal Loss).
FINAL_WEIGHTS = ART_DIR / "best_mlp_medium_FocalLoss_alpha075_gamma20_run12.pth"
score_model = MediumMLP(in_dim=X_valid_proc.shape[1], dropout=0.30).to(DEVICE)
score_model.load_state_dict(torch.load(FINAL_WEIGHTS, map_location=DEVICE))
score_model.eval()

# Próg decyzyjny wyznaczony na zbiorze walidacyjnym (max F1) - bez podglądania t
thr_score, _ = best_f1_threshold(y_valid.values, predict_proba(score_model, X_va
print(f"Próg decyzyjny (max F1 na valid): {thr_score:.4f}")

# Scoring nowych danych -> tabela scoringowa
score_table = score_dataset(score_input, score_model, preprocessor, thr_score)
score_table.head(15)

```

Zbiór scoringowy: (118108, 433) (kolumna 'isFraud' usunięta: True)
 Próg decyzyjny (max F1 na valid): 0.4741

Out[17]:

	TransactionID	P_isFraud0	P_isFraud1	EM_EVENTPROBABILITY	EM_CLASSIFICATION
0	3459432	0.741621	0.258379	0.258379	0
1	3459433	0.830695	0.169305	0.169305	0
2	3459434	0.644912	0.355087	0.355087	0
3	3459435	0.525012	0.474988	0.474988	1
4	3459436	0.715330	0.284670	0.284670	0
5	3459437	0.815648	0.184352	0.184352	0
6	3459438	0.805592	0.194408	0.194408	0
7	3459439	0.757079	0.242921	0.242921	0
8	3459440	0.741977	0.258023	0.258023	0
9	3459441	0.822966	0.177034	0.177034	0
10	3459442	0.775154	0.224846	0.224846	0
11	3459443	0.800269	0.199731	0.199731	0
12	3459444	0.767763	0.232237	0.232237	0
13	3459445	0.761880	0.238120	0.238120	0
14	3459446	0.775897	0.224103	0.224103	0



In [18]:

```
n = len(score_table)
n_fraud = int((score_table["EM_CLASSIFICATION"] == 1).sum())
print(f"Zeskorowanych rekordów : {n:,}")
print(f"Oznaczone jako Fraud : {n_fraud:,} ({n_fraud / n:.2%})")
print(f"Oznaczone jako Legit : {n - n_fraud:,} ({(n - n_fraud) / n:.2%})")
print(f"Średnie P(isFraud) : {score_table['EM_EVENTPROBABILITY'].mean():.4f}")
print(f"Mediana P(isFraud) : {score_table['EM_EVENTPROBABILITY'].median():.4f}")

# Eksport tabeli scoringowej (analog "Eksport tabeli scoringowej" z SAS EM, slaj
SCORE_OUT = ART_DIR / "score_table_oot.csv"
score_table.to_csv(SCORE_OUT, index=False)
print(f"\nZapisano tabelę scoringową: {SCORE_OUT}")
```

Zeskorowanych rekordów : 118,108
 Oznaczone jako Fraud : 3,065 (2.60%)
 Oznaczone jako Legit : 115,043 (97.40%)
 Średnie P(isFraud) : 0.2184
 Mediana P(isFraud) : 0.1948

Zapisano tabelę scoringową: artifacts_mlp\score_table_oot.csv